DBMS (Databases) - తెలుగు గైడ్ (SSE Fundamentals)

> ఈ document చదివిన తర్వాత database concepts మళ్ళీ జీవితంలో మర్చిపోలేవు. Database theory అస్సలు తెలియని వ్యక్తిని — ZERO నుండి — Senior Software Engineer (SSE) interview crack చేసే స్థాయికి తీసుకెళ్లడమే లక్ష్యం. ప్రతి concept కి ఒక vivid real-life analogy, ఎందుకు/ఎప్పుడు వాడాలి, internal working, trade-offs, gotchas, మరియు runnable **SQL** code — అన్నీ ఉంటాయి. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."
>
> **నీ background:** నువ్వు self-taught MERN developer — MongoDB/Mongoose తెలుసు, కొంచెం SQL చూసి ఉండొచ్చు, కానీ **database theory (relational model, normalization, transactions, indexing, ACID) అస్సలు తెలియదు.** Non-CS background. ఈ guide అదే gap ని పూరిస్తుంది. నీకు Mongo తెలుసు కాబట్టి, ప్రతి చోట **relational vs document, SQL vs NoSQL** connections చూపిస్తా — దీంతో కొత్త concept నీ existing knowledge కి అతుక్కుంటుంది.
>
> **లక్ష్యం:** Best-teacher style — intuition first, formalism తర్వాత. Absolute basics నుండి SSE interview depth వరకు. SQL నేర్చుకోవడానికి ఉత్తమ మార్గం చేసి *చూడటం* — అందుకే ప్రతి చోట runnable SQL ఇచ్చాను, నువ్వు db-fiddle.com లేదా local PostgreSQL/MySQL లో paste చేసి run చేయవచ్చు.
>
> Scaling (replication, sharding, CAP) లోతుగా `HLD_Go_Telugu.md` / `SystemDesign_Go_Telugu.md` లలో ఉంది; ఇక్కడ database-centric view ఇస్తా.

విషయ సూచిక (Table of Contents)

Part 1 — Basics (పునాదులు)

1. DBMS అంటే ఏమిటి, ఎందుకు (file system vs DBMS, RDBMS, 3-schema architecture, data independence)
2. ER Model (entities, attributes, relationships 1:1/1:N/M:N, ER diagram → tables)
3. Relational Model & Keys (tables/tuples/attributes, super/candidate/primary/foreign/composite keys, constraints, referential integrity)

Part 2 — SQL (deep, hands-on)

4. SQL Basics (DDL/DML/DCL/TCL, CREATE/INSERT/UPDATE/DELETE, data types)
5. Querying (SELECT/WHERE/ORDER BY/LIMIT/DISTINCT/LIKE/IN/BETWEEN/operators)
6. Joins (INNER/LEFT/RIGHT/FULL/CROSS/SELF — running example + diagrams)
7. Aggregation & Advanced (GROUP BY/HAVING/COUNT/SUM/AVG, subqueries, CTEs, window functions)

Part 3 — Design (డిజైన్)

8. Functional Dependencies & Normalization (anomalies, 1NF→2NF→3NF→BCNF, when to denormalize)

Part 4 — Transactions (లావాదేవీలు)

9. Transactions & ACID (deep, with examples)
10. Concurrency Control (dirty/non-repeatable/phantom reads, locks, 2PL, isolation levels)

11. Deadlocks in DB (detection, prevention, avoidance)

Part 5 — Performance (పనితీరు)

12. Indexing (B-tree/B+tree, clustered vs non-clustered, composite, covering, EXPLAIN)
13. Query Optimization & Storage (execution plan, pages/buffer pool, WAL)

Part 6 — Modern (ఆధునికం)

14. SQL vs NoSQL (relational vs document/MongoDB, when which, other NoSQL types)
15. Scaling: Replication, Sharding, CAP Theorem (brief)
16. Interview Q&A + Memory Tips + Common Mistakes

Part 1 — Basics (పునాదులు)

> ఈ Part లో database అంటే అసలు ఏమిటి, ఎందుకు అవసరం (plain files తో ఎందుకు సరిపోదు), data ని ఎలా design చేయాలి (ER model), మరియు relational world లో అన్నిటికీ ఆధారమైన **keys** — ఇవి నేర్చుకుంటాం. ఇవి పునాది; ఇవి పక్కాగా ఉంటే SQL, normalization, transactions అన్నీ సులభం.

---

1. DBMS అంటే ఏమిటి, ఎందుకు

వివరణ

Data = ముడి facts (raw facts). ఉదా: "Ravi", "9876543210", "₹500". వీటికి context లేకపోతే అర్థం లేదు.

Database = organized గా, related గా store చేసిన data collection. ఉదా: ఒక e-commerce app లో customers, orders, products అన్నీ ఒక చోట, ఒక structure లో.

DBMS (Database Management System) = ఆ database ని create, read, update, delete (CRUD), secure, backup, query చేయడానికి వాడే **software**. ఉదా: MySQL, PostgreSQL, Oracle, SQL Server, **MongoDB**, Redis.

అంటే: **database = data + structure; DBMS = దాన్ని manage చేసే software.** రోజువారీ మాటలో "database" అంటే చాలామంది DBMS ని కూడా కలిపి అంటారు — పర్వాలేదు, context తో అర్థమవుతుంది.

నీకు ఇప్పటికే MongoDB తెలుసు కాబట్టి — **MongoDB కూడా ఒక DBMS.** అది document-oriented (NoSQL). ఈ guide ఎక్కువగా **RDBMS (Relational DBMS)** గురించి, ఎందుకంటే interview theory అంతా అక్కడ నుండే వస్తుంది, మరియు relational fundamentals తెలిస్తే Mongo ని కూడా better గా వాడతావు.

Real-life Scenario

> **DBMS = ఒక పెద్ద library + librarian.**
>
> ఊహించు — నీ దగ్గర 10 లక్షల పుస్తకాలు ఉన్నాయి.
>
> - **విధానం 1 (file system):** పుస్తకాలన్నీ ఒక పెద్ద గదిలో గుమ్మరించావు. ఒక పుస్తకం కావాలంటే గంటల తరబడి వెతకాలి. రెండు మంది ఒకేసారి అదే పుస్తకం మార్చాలంటే గొడవ. ఒక పుస్తకం title రెండు చోట్ల రాసి ఉంటే, ఒకటి update చేసి రెండోది మర్చిపోతే — inconsistency.
> - **విధానం 2 (DBMS):** ఒక **librarian (DBMS)** ఉన్నాడు. పుస్తకాలు shelf ల్లో indexed గా ఉన్నాయి (index). "ఈ author పుస్తకాలు ఇవ్వు" అంటే వెంటనే తెస్తాడు (query). ఇద్దరు ఒకేసారి request చేస్తే గొడవ లేకుండా handle చేస్తాడు (concurrency). ఒకరికి restricted section access లేదు (security). రోజూ backup తీస్తాడు (recovery).
>
> **File system = గుమ్మరించిన గది; DBMS = librarian + organized library.** అదే తేడా.

File System (plain files) తో ఎందుకు సరిపోదు?

Application data ని plain files (CSV, JSON, text) లో store చేస్తే వచ్చే సమస్యలు — interview లో ఇవి తప్పకుండా అడుగుతారు:

సమస్య	File System లో	DBMS ఎలా solve చేస్తుంది
Data Redundancy (డేటా పునరావృతం)	అదే customer address 5 files లో copy → చోటు వృథా	Normalization + relationships → ఒకేసారి store
Data Inconsistency (అస్థిరత)	ఒక చోట address update, మిగతా చోట్ల పాతదే → mismatch	Single source of truth + foreign keys
Difficult Access	"₹1000+ orders ఇచ్చిన Bangalore customers" → custom program రాయాలి	SQL query ఒక్క line
No Concurrency Control	ఇద్దరు ఒకేసారి write → data corrupt	Locks, transactions, isolation
No Security	file access ఉంటే అంతా కనిపిస్తుంది	User-level, row/column-level permissions (GRANT)
No Integrity	age = -5, invalid email store అవుతుంది	Constraints (CHECK, NOT NULL, FK)
No Recovery	crash అయితే data పోతుంది	Transactions + WAL + backup
No Standard Query	ప్రతి app సొంత logic	SQL — universal standard

అందుకే serious application ఏదైనా DBMS వాడుతుంది, plain files కాదు.

RDBMS అంటే ఏమిటి?

RDBMS (Relational Database Management System) = data ని **tables** (rows + columns) రూపంలో store చేసే DBMS. Tables మధ్య **relationships** (foreign keys) ఉంటాయి. ఇది **E.F. Codd (1970)** ప్రతిపాదించిన **relational model** మీద ఆధారపడి ఉంటుంది.

ఉదా: ఒక `customers` table, ఒక `orders` table. ప్రతి order ఏ customer దో `customer_id` foreign key తో link అవుతుంది.

```
customers                          orders
+----+---------+-----------+       +----+-------------+--------+
| id | name    | city      |       | id | customer_id | amount |
+----+---------+-----------+       +----+-------------+--------+
| 1  | Ravi    | Bangalore |       | 101|      1      |  500   |
| 2  | Sita    | Hyderabad |       | 102|      1      |  300   |
+----+---------+-----------+       | 103|      2      |  900   |
                                   +----+-------------+--------+
      ↑ id = primary key                   ↑ customer_id = foreign key (→ customers.id)
```

Popular RDBMS: **MySQL, PostgreSQL, Oracle, Microsoft SQL Server, SQLite.** అన్నీ **SQL (Structured Query Language)** వాడతాయి — ఒకే language, dialect తేడాలు మాత్రమే.

Mongo తో pole: Mongo లో "table" ని **collection** అంటారు, "row" ని **document** (JSON-like), "column" ని **field** అంటారు. Mongo schema-flexible — ప్రతి document వేరే fields కలిగి ఉండొచ్చు. RDBMS లో schema fixed — ప్రతి row అదే columns కలిగి ఉంటుంది. ఈ తేడా Topic 14 లో లోతుగా చూద్దాం.

3-Schema Architecture (ANSI-SPARC)

DBMS ని **3 levels** గా విభజిస్తారు. ఇది ఎందుకు ముఖ్యం? ఎందుకంటే ఇది **abstraction** ఇస్తుంది — data ఎలా store అవుతుందో (physical) తెలియకుండా users query చేయవచ్చు.

```
┌──────────────────────────────────────────────┐
│  EXTERNAL LEVEL (View level)                  │  ← ప్రతి user ఏం చూస్తాడో
│  ఉదా: sales team కి customer name+order మాత్రమే│    (multiple views)
│       HR కి salary కనిపిస్తుంది                    │
├──────────────────────────────────────────────┤
│  CONCEPTUAL / LOGICAL LEVEL                   │  ← మొత్తం DB structure
│  ఏ tables, columns, relationships, constraints│    (ఒకటే logical schema)
├──────────────────────────────────────────────┤
│  INTERNAL / PHYSICAL LEVEL                    │  ← disk లో ఎలా store
│  files, pages, indexes, compression, B+trees  │    (storage details)
└──────────────────────────────────────────────┘
```

Level	ఏం describe చేస్తుంది	ఎవరికి సంబంధించింది
External (View)	ప్రతి user/app కి కనిపించే subset. Views, permissions	End users, apps
Conceptual (Logical)	మొత్తం database — tables, columns, types, relationships	DB designers, developers
Internal (Physical)	Disk మీద actual storage — files, pages, indexes	DB engine, DBA

Data Independence — దీని పెద్ద benefit

Data Independence = ఒక level మార్చినా, పై level ని మార్చకుండా ఉండగలగడం. రెండు రకాలు:

- **Physical Data Independence:** Internal level (storage) మార్చినా — ఉదా: కొత్త index add చేసినా, storage format మార్చినా — **conceptual/logical schema మారదు.** Query లు అలాగే పనిచేస్తాయి. ఇది **సులభం**, ఎక్కువగా achieve అవుతుంది.
- **Logical Data Independence:** Conceptual level (tables/columns) మార్చినా — ఉదా: కొత్త column add చేసినా — **external views/apps మారవు** (ideally). ఇది **కష్టం**, ఎందుకంటే apps directly schema మీద ఆధారపడతాయి.

> **Analogy:** నువ్వు car నడుపుతున్నావు (external view — steering, brake). లోపల engine petrol నుండి electric కి మార్చినా (internal change), నీ driving experience మారదు — అదే physical data independence. కానీ dashboard layout మారితే (logical change) నువ్వు కొంచెం adjust అవ్వాలి — అది logical independence, harder.

Key Points

- **Data = raw facts; Database = organized data; DBMS = manage చేసే software.** MongoDB కూడా DBMS.

- **File system సమస్యలు:** redundancy, inconsistency, no concurrency, no security, no integrity, no recovery, no standard query — DBMS ఇవన్నీ solve చేస్తుంది. (Interview favorite.)
- **RDBMS** = tables + relationships, SQL వాడుతుంది, Codd's relational model (1970) ఆధారం.
- **3-schema architecture:** External (views) → Conceptual (logical schema) → Internal (physical storage). Abstraction ఇస్తుంది.
- **Data independence:** Physical (storage మార్పు → logical మారదు, easy) vs Logical (schema మార్పు → views మారవు, hard).

Interview దృష్టి

Q: File system ఉండగా DBMS ఎందుకు? A: File system లో data redundancy, inconsistency, no concurrent access control, no security, no integrity constraints, no crash recovery, మరియు ప్రతి query కి custom program అవసరం. DBMS ఇవన్నీ centralized గా handle చేస్తుంది — normalization, transactions, locks, GRANT, constraints, WAL, SQL. అందుకే multi-user, reliable applications కి DBMS తప్పనిసరి.

Q: 3-schema architecture యొక్క ఉద్దేశ్యం ఏమిటి? A: **Abstraction & data independence.** Users physical storage details తెలియకుండా data వాడగలరు. Storage మార్చినా (physical independence) apps break అవ్వవు. ఇది maintainability, security (per-user views), flexibility ఇస్తుంది.

Q: MongoDB RDBMS నా? A: కాదు. MongoDB ఒక **NoSQL, document-oriented DBMS** — data ని BSON documents (collections) లో store చేస్తుంది, tables/rows కాదు. Schema-flexible, joins బదులు embedding. కానీ ఇది కూడా DBMS — CRUD, indexing, concurrency, replication అన్నీ ఉంటాయి. (Topic 14 లో deep dive.)

2. ER Model (Entity-Relationship Model)

వివరణ

Table లు రాయడం మొదలుపెట్టే ముందు, **"నా data ఎలా ఉండాలి?"** అని design చేయాలి. ఈ design ని కాగితం మీద గీయడానికి వాడే tool = **ER Model (Entity-Relationship Model)**, Peter Chen (1976) ప్రతిపాదించాడు. ఇది database యొక్క **blueprint** — ఇల్లు కట్టే ముందు గీసే plan లాంటిది.

ER model లో 3 ముఖ్య భాగాలు:

1. Entity (వస్తువు): real-world లో ఒక distinct object/concept. ఉదా: `Student`, `Course`, `Customer`, `Order`. ప్రతి entity తర్వాత ఒక **table** అవుతుంది. ER diagram లో **rectangle (దీర్ఘచతురస్రం)**.

- **Strong entity:** సొంత primary key ఉంటుంది (ఉదా: Customer — customer_id).
- **Weak entity:** సొంతగా unique గా identify అవ్వలేదు; మరో (owner) entity మీద ఆధారపడుతుంది. ఉదా: `OrderItem` ఒక `Order` లేకుండా అర్థం లేదు. Double rectangle తో గీస్తారు.

2. Attribute (లక్షణం): entity యొక్క property. ఉదా: Student కి `name`, `roll_no`, `age`. ER diagram లో **ellipse (దీర్ఘవృత్తం)**. Table లో ఇది **column** అవుతుంది. రకాలు:

Attribute రకం	అర్థం	ఉదాహరణ
Simple (atomic)	ఇంకా విభజించలేనిది	age, roll_no
Composite	చిన్న parts గా విడగొట్టవచ్చు	name → (first, last); address → (street, city, pin)
Single-valued	ఒకటే value	date_of_birth
Multi-valued	అనేక values	phone_numbers (ఒకరికి 2-3 phones) — double ellipse
Derived	వేరే attribute నుండి లెక్కించేది	age (dob నుండి derive) — dashed ellipse
Key attribute	unique గా identify చేసేది	roll_no — underlined

3. Relationship (సంబంధం): రెండు (లేదా అంతకంటే ఎక్కువ) entities మధ్య association. ఉదా: Student **enrolls in** Course; Customer **places** Order. ER diagram లో **diamond (రాంబస్)**.

Real-life Scenario

> **ER model = పెళ్లి సంబంధాల చార్ట్ (family tree).**
> ఒక కుటుంబం గీయాలనుకో:
>
> - **Entities** = వ్యక్తులు (Person) — ప్రతి ఒక్కరూ ఒక rectangle.
> - **Attributes** = ప్రతి వ్యక్తి లక్షణాలు — పేరు, వయసు, ఊరు.
> - **Relationships** = వాళ్ళ మధ్య సంబంధాలు — "married to", "parent of", "sibling of".
>
> ఒక తండ్రికి **చాలా మంది** పిల్లలు ఉండొచ్చు (1:N). ఒక భర్తకి **ఒకే** భార్య (1:1). ఒక event కి **చాలామంది** వస్తారు, ఒక వ్యక్తి **చాలా** events కి వెళ్తాడు (M:N).
> ఈ చార్ట్ గీస్తే, ఎవరు ఎవరితో ఎలా connected అనేది ఒక్క చూపులో అర్థమవుతుంది. **Database design కూడా అంతే — entities గీసి, relationships connect చేస్తే schema తయారు.**

Cardinality — relationships యొక్క గుండె (1:1, 1:N, M:N)

Cardinality = ఒక entity యొక్క ఎన్ని instances మరో entity యొక్క ఎన్ని instances తో connect అవుతాయో చెప్పేది. ఇది database design లో అత్యంత ముఖ్యమైన decision — interview లో తప్పకుండా అడుగుతారు.

రకం	అర్థం	ఉదాహరణ	Table లో ఎలా
One-to-One (1:1)	A యొక్క ఒక్క instance ↔ B యొక్క ఒక్క instance	ఒక Person ↔ ఒక Passport	Foreign key ఏదో ఒక side; లేదా రెండూ కలిపి ఒకే table
One-to-Many (1:N)	A యొక్క ఒకటి ↔ B యొక్క చాలా	ఒక Customer → చాలా Orders	"many" side (Order) లో foreign key (customer_id)
Many-to-Many (M:N)	A చాలా ↔ B చాలా	Students ↔ Courses (ఒక student చాలా courses, ఒక course కి చాలా students)	**Junction/bridge table** అవసరం

```
1:1     Person ────────── Passport         (each side one)

1:N     Customer ──────< Orders            (crow's foot = many)
          1               N

M:N     Students >──────< Courses          (both many → junction table)
```

ER Diagram → Tables (mapping rules) — అత్యంత ముఖ్యం

ER diagram గీసాక, దాన్ని actual tables గా ఎలా మార్చాలి? ఇవి **mapping rules** — interview లో practical question.

Rule 1 — ప్రతి strong entity → ఒక table. Attributes → columns, key attribute → primary key.

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    city        VARCHAR(50)
);
```

Rule 2 — 1:N relationship → "many" side లో foreign key. (కొత్త table అవసరం లేదు.)

```sql
-- ఒక customer కి చాలా orders → order (many side) లో customer_id పెట్టు
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),  -- FK
    amount      DECIMAL(10,2)
);
```

Rule 3 — M:N relationship → కొత్త junction table. ఇది రెండు FK లు కలిగి ఉంటుంది (వాటి composite = primary key).

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(100)
);
CREATE TABLE courses (
    course_id  INT PRIMARY KEY,
    title      VARCHAR(100)
);
-- M:N → junction table (దీనికి enrollment అని పేరు)
CREATE TABLE enrollments (
    student_id INT REFERENCES students(student_id),
    course_id  INT REFERENCES courses(course_id),
    grade      CHAR(2),                          -- relationship attribute!
    PRIMARY KEY (student_id, course_id)          -- composite key
);
```

> **గమనిక:** M:N లో relationship కి కూడా attribute ఉండొచ్చు — ఉదా: enrollment కి `grade` , `enrolled_date` . అవి junction table లోకి వెళ్తాయి. ఇది beginners మర్చిపోయే point.

Rule 4 — Multi-valued attribute → separate table. ఉదా: ఒక customer కి చాలా phone numbers.

```
CREATE TABLE customer_phones (
    customer_id INT REFERENCES customers(customer_id),
    phone       VARCHAR(15),
    PRIMARY KEY (customer_id, phone)
);
```

Rule 5 — 1:1 → foreign key ఏదో ఒక side (ఎక్కువ optional ఉన్న side లో), UNIQUE constraint తో.

```
CREATE TABLE passports (
    passport_id INT PRIMARY KEY,
    person_id   INT UNIQUE REFERENCES persons(person_id),  -- UNIQUE → 1:1
    number      VARCHAR(20)
);
```

Mongo తో pole: Mongo లో 1:N ని రెండు రకాలుగా చేయవచ్చు — **embedding** (orders array ని customer document లోపల పెట్టడం) లేదా **referencing** (customer_id store చేయడం, SQL లాగా). M:N కి referencing వాడతారు. అంటే ER thinking Mongo కి కూడా వర్తిస్తుంది — కానీ Mongo లో junction table బదులు embedded arrays వాడొచ్చు. Design decision Topic 14 లో.

Full ER Example — E-commerce

```
                places                    contains                (in)
             1    N                  1    N                 N   1
┌──────────┐          ┌─────────┐          ┌───────────┐         ┌──────────┐
│ Customer │─────────<│  Order  │─────────<│ OrderItem │>────────│ Product  │
└──────────┘          └─────────┘          └───────────┘         └──────────┘

customer_id(PK)       order_id(PK)          order_id(FK)          product_id(PK)
name                  customer_id(FK)       product_id(FK)        name
city                  order_date            quantity              price
                      status                                      stock
```

ఇక్కడ Customer↔Order = **1:N**. Order↔Product = **M:N** (ఒక order లో చాలా products, ఒక product చాలా orders లో), అందుకే `OrderItem` = junction table (దీనికి `quantity` అనే relationship attribute కూడా ఉంది).

Key Points

- **ER model = database blueprint.** Entity (rectangle → table), Attribute (ellipse → column), Relationship (diamond).
- **Attribute రకాలు:** simple, composite, single/multi-valued, derived, key. Multi-valued → separate table.
- **Cardinality:** 1:1, 1:N, M:N. ఇదే most important design decision.
- **Mapping rules:** strong entity → table; **1:N → FK on many side**; **M:N → junction table**; multi-valued → separate table; 1:1 → FK + UNIQUE.
- **Weak entity** (OrderItem) కి owner entity అవసరం, సొంత key ఉండదు — composite key వాడుతుంది.

Interview దృష్టి

Q: M:N relationship ని tables లో ఎలా represent చేస్తావు? A: రెండు entities ని directly connect చేయలేం (redundancy వస్తుంది). అందుకే **junction (bridge/associative) table** create చేస్తా — ఇది రెండు entities యొక్క foreign keys కలిగి ఉంటుంది, వాటి **composite = primary key**. Relationship కి attributes ఉంటే (ఉదా: enrollment grade) అవి కూడా ఈ junction table లోకి వెళ్తాయి. ఉదా: students ↔ courses → enrollments table.

Q: 1:N relationship లో foreign key ఏ side పెడతావు, ఎందుకు? A: ఎప్పుడూ **"many" side లో.** ఉదా: ఒక customer కి చాలా orders → `customer_id` ని `orders` table లో పెడతా. ఎందుకంటే ప్రతి order కి exactly ఒక customer ఉంటాడు (single value → column లో పడుతుంది). "One" side లో పెడితే multiple order_ids ని ఒక column లో పెట్టాలి — అది 1NF violate చేస్తుంది.

Q: Weak entity అంటే ఏమిటి? A: సొంతగా unique గా identify అవ్వలేని entity. దానికి **partial key** మాత్రమే ఉంటుంది; owner (strong) entity యొక్క key తో కలిపి full key అవుతుంది. ఉదా: `OrderItem` ఒక `Order` లేకుండా ఉనికి లేదు — దాని key = (order_id + product_id).

3. Relational Model & Keys

వివరణ

Relational Model (Codd, 1970) = data ని **relations (tables)** గా చూసే mathematical model. దీని terminology interview లో అడుగుతారు — formal పదాలు తెలుసుకో:

Formal పదం	సాధారణ పదం	Mongo లో	అర్థం
Relation	Table	Collection	మొత్తం table
Tuple	Row / Record	Document	ఒక్క వరుస (ఒక entity instance)
Attribute	Column / Field	Field	ఒక్క column
Degree	—	—	columns సంఖ్య
Cardinality	—	—	rows సంఖ్య
Domain	—	—	ఒక column కి valid values set (ఉదా: age → 0-150)
Relation schema	Table structure	—	table definition (columns + types)
Relation instance	Table data	—	ఒక క్షణంలో table లోని actual rows

Relational model యొక్క properties (rules):

1. ప్రతి row **unique** గా ఉండాలి (duplicate rows ఉండకూడదు — key తో ensure).
2. Column values **atomic** (indivisible) గా ఉండాలి — ఒక cell లో list/array కాదు (ఇది 1NF).
3. Rows/columns యొక్క **order matter చేయదు** — set లాంటిది.
4. ప్రతి column కి unique పేరు, ఒక domain (type).

Real-life Scenario

> **Key = నీ Aadhaar number.**
>
> ఒక ఊర్లో "Ravi" అనే పేరున్న 500 మంది ఉండొచ్చు. పేరుతో ఒక వ్యక్తిని unique గా గుర్తించలేం. కానీ **Aadhaar number** ప్రతి ఒక్కరికీ unique — అది చెప్తే exactly ఒక వ్యక్తి.
>
> - నీ (Aadhaar) — ఒక్కటే నిన్ను identify చేస్తుంది → **primary key.**
> - నిన్ను identify చేయగల అన్నీ (Aadhaar, PAN, phone) → **candidate keys.**
> - Aadhaar + పేరు + address కలిపి కూడా identify చేస్తుంది, కానీ అనవసరంగా ఎక్కువ → **super key.**
> - నీ bank account form లో నీ Aadhaar ని రాస్తే — ఆ form నిన్ను reference చేస్తోంది → **foreign key.**
>
> Key అంటే **"ఏ column(s) ఒక row ని unique గా గుర్తిస్తాయి"** అనే ప్రశ్నకు జవాబు. Relational model మొత్తం keys మీదే నిలబడుతుంది.

Keys యొక్క రకాలు — depth తో

ఒక example table తీసుకుందాం: `employees(emp_id, email, pan, name, dept_id)` .

1. Super Key: ఒక row ని unique గా identify చేసే **ఏ column(s) set అయినా.** అనవసర columns ఉన్నా పర్వాలేదు.

- ఉదా: `{emp_id}` , `{emp_id, name}` , `{email}` , `{email, name}` — అన్నీ super keys (ఎందుకంటే emp_id/email అప్పటికే unique).

2. Candidate Key: minimal super key — దీనిలో ఏ column తీసేసినా అది ఇక unique గా identify చేయదు. అంటే "అనవసర columns లేని" super key.

- ఉదా: `{emp_id}` , `{email}` , `{pan}` — ఇవి candidate keys. `{emp_id, name}` candidate key **కాదు** (name అనవసరం, తీసేసినా emp_id unique).
- ఒక table కి **చాలా candidate keys** ఉండొచ్చు.

3. Primary Key (PK): candidate keys లో మనం **ఎంచుకున్న ఒక్కటి** — main identifier. Rules: **NOT NULL + UNIQUE.** ఒక table కి **ఒకటే** primary key.

- ఉదా: emp_id ని primary key గా ఎంచుకున్నాం. (email, pan మిగిలిన candidate keys → "alternate keys.")

4. Alternate Key: primary key గా ఎంచుకోని candidate keys. (email, pan ఇక్కడ.)

5. Composite Key: ఒకటి కంటే ఎక్కువ columns కలిపి ఏర్పడే key. Junction tables లో common.

- ఉదా: `enrollments(student_id, course_id)` → PK = (student_id, course_id).

6. Foreign Key (FK): ఒక table లోని column, అది **మరో table యొక్క primary key ని reference** చేస్తుంది. ఇదే tables ని link చేసేది.

- ఉదా: `orders.customer_id` → `customers.customer_id` ని reference చేస్తుంది.

7. Surrogate Key: business అర్థం లేని, system-generated artificial key (auto-increment id, UUID). vs **Natural Key** (business-meaningful — email, PAN). ఎక్కువగా surrogate keys ప్రాధాన్యం (stable, small, fast).

```
super key ⊇ candidate key ⊇ primary key
(అతి పెద్ద set → minimal → ఎంచుకున్నది)

{emp_id, name, email}   ← super key (extra columns)
        ↓ minimize
{emp_id}, {email}, {pan} ← candidate keys (minimal)
        ↓ choose one
{emp_id}                 ← primary key
```

Code — keys, constraints ఆచరణలో

```
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id   INT PRIMARY KEY,               -- primary key (NOT NULL + UNIQUE auto)
    email    VARCHAR(100) NOT NULL UNIQUE,  -- candidate/alternate key
    pan      CHAR(10) UNIQUE,               -- మరో candidate key
    name     VARCHAR(100) NOT NULL,         -- NOT NULL constraint
    age      INT CHECK (age >= 18),         -- CHECK constraint
    salary   DECIMAL(10,2) DEFAULT 0,       -- DEFAULT constraint
    dept_id  INT,
    -- foreign key: dept_id తప్పకుండా departments లో ఉన్న dept_id అవ్వాలి
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- composite primary key ఉదాహరణ (junction table)
CREATE TABLE project_members (
    emp_id     INT REFERENCES employees(emp_id),
    project_id INT,
    role       VARCHAR(30),
    PRIMARY KEY (emp_id, project_id)        -- composite key
);
```

Constraints (నియంత్రణలు) — data integrity యొక్క రక్షకులు

Constraints = data లోకి invalid values రాకుండా ఆపే rules. DBMS వీటిని enforce చేస్తుంది — application code మర్చిపోయినా DB కాపాడుతుంది.

Constraint	ఏం చేస్తుంది	ఉదాహరణ
NOT NULL	value తప్పకుండా ఉండాలి, NULL కాదు	`name VARCHAR NOT NULL`
UNIQUE	ఆ column లో duplicate values ఉండకూడదు	`email UNIQUE`
PRIMARY KEY	NOT NULL + UNIQUE కలిపి	`emp_id INT PRIMARY KEY`
FOREIGN KEY	value మరో table యొక్క PK లో ఉండాలి	`REFERENCES departments(dept_id)`
CHECK	condition satisfy అవ్వాలి	`CHECK (age >= 18)`
DEFAULT	value ఇవ్వకపోతే default వాడు	`status VARCHAR DEFAULT 'active'`

Referential Integrity — foreign keys యొక్క నియమం

Referential Integrity = ఒక foreign key value **ఎప్పుడూ** referenced table లో ఉండాలి (లేదా NULL). అంటే "orphan" rows ఉండకూడదు.

- ఉదా: `orders.customer_id = 5` ఉంటే, `customers` లో customer_id = 5 తప్పకుండా ఉండాలి. లేని customer కి order ఉండకూడదు.

మరి referenced row ని **delete/update చేస్తే** ఏమవుతుంది? ఇక్కడే **referential actions** వస్తాయి — SSE interview favorite:

Action	delete/update చేసినప్పుడు ఏమవుతుంది
CASCADE	Parent delete అయితే → child rows కూడా delete. (customer delete → అతని orders కూడా delete)
SET NULL	Parent delete అయితే → child లో FK = NULL
SET DEFAULT	Parent delete అయితే → child FK = default value
RESTRICT / NO ACTION	Child rows ఉంటే parent ని delete **చేయనివ్వదు** (error). Default behavior.

```
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT,
    amount      DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        ON DELETE CASCADE      -- customer పోతే, orders కూడా పోతాయి
        ON UPDATE CASCADE      -- customer_id మారితే, ఇక్కడ కూడా update
);
```

> **గమనిక:** `ON DELETE CASCADE` శక్తివంతమైనది కానీ ప్రమాదకరం — పొరపాటున ఒక customer delete చేస్తే వాళ్ళ order history మొత్తం పోతుంది. Financial/audit data కి `RESTRICT` లేదా soft-delete (a `deleted_at` column) safer.

Key Points

- **Terminology:** relation=table, tuple=row, attribute=column, degree=#columns, cardinality=#rows, domain=valid values.
- **Keys hierarchy:** super key (⊇) → candidate key (minimal) → primary key (ఎంచుకున్నది). Alternate = మిగిలిన candidate keys.
- **Primary key = NOT NULL + UNIQUE, ఒక్కటే per table.** Composite = multiple columns. Foreign key links tables.
- **Surrogate (auto-id) vs natural (email/PAN):** surrogate ఎక్కువగా preferred — stable, small, fast.
- **Constraints:** NOT NULL, UNIQUE, PK, FK, CHECK, DEFAULT — DB-level data integrity.
- **Referential integrity:** FK ఎప్పుడూ valid PK ని point చేయాలి. Actions: CASCADE, SET NULL, RESTRICT.

Interview దృష్టి

Q: Super key, candidate key, primary key తేడా? A: **Super key** = row ని unique గా identify చేసే ఏ column set (extra columns ok). **Candidate key** = minimal super key (ఏ column తీసేసినా uniqueness పోతుంది). **Primary key** = candidate keys లో మనం main గా ఎంచుకున్న ఒక్కటి, NOT NULL + UNIQUE. అంటే super key ⊇ candidate key, primary key ∈ candidate keys.

Q: Primary key vs Unique key తేడా? A: (1) Primary key **NOT NULL**; unique key **ఒక NULL** allow చేస్తుంది (SQL standard). (2) ఒక table కి **ఒకటే** primary key, కానీ **చాలా** unique keys. (3) Primary key తరచుగా clustered index అవుతుంది. రెండూ uniqueness enforce చేస్తాయి.

Q: Surrogate key ఎందుకు ప్రాధాన్యం? A: Natural keys (email, phone) మారవచ్చు (email మార్చొచ్చు) → అన్ని FK లు update అవ్వాలి, పెద్ద తలనొప్పి. Surrogate (auto-increment/UUID) ఎప్పటికీ మారదు, small (fast joins/indexes), business logic మీద ఆధారపడదు. అందుకే production లో ఎక్కువగా surrogate PK + natural columns మీద UNIQUE constraint వాడతారు.

Q: ON DELETE CASCADE ఎప్పుడు వాడకూడదు? A: Data loss risky ఉన్న చోట — financial records, audit logs, user history. పొరపాటున parent delete చేస్తే children silently పోతాయి. అలాంటి చోట్ల RESTRICT (delete ఆపు) లేదా soft-delete pattern (deleted_at flag) better.

Part 2 — SQL (deep, hands-on)

> ఇక్కడే అసలు పని మొదలు. SQL (Structured Query Language) = relational databases తో మాట్లాడే language. దీన్ని **చదవడం వల్ల నేర్చుకోలేవు — చేయడం వల్లే నేర్చుకుంటావు.** ప్రతి code block ని db-fiddle.com (PostgreSQL 15 ఎంచుకో) లో paste చేసి run చేయి. ఈ Part అంతా ఒక **e-commerce schema** (customers, orders, products, order_items, employees, departments) మీద నడుస్తుంది — ఒకసారి setup చేసుకుంటే అన్ని topics కి అదే వాడొచ్చు.

---

4. SQL Basics (DDL/DML/DCL/TCL)

వివరణ

SQL = Structured Query Language. 1974 లో IBM (SEQUEL) నుండి పుట్టింది, ఇప్పుడు ANSI/ISO standard. ప్రతి RDBMS (MySQL, PostgreSQL, Oracle...) దీన్ని వాడుతుంది — syntax లో చిన్న dialect తేడాలు మాత్రమే.

SQL commands ని **4 categories** గా విభజిస్తారు — interview లో ఇది classic question:

Category	పూర్తి పేరు	ఏం చేస్తుంది	Commands
DDL	Data **Definition** Language	Structure (schema) define/మార్చడం	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML	Data **Manipulation** Language	Data ని add/change/remove	`INSERT`, `UPDATE`, `DELETE`, (`SELECT` *)
DCL	Data **Control** Language	Permissions (access control)	`GRANT`, `REVOKE`
TCL	**Transaction** Control Language	Transactions manage	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

(SELECT ని కొందరు DQL — Data Query Language గా విడిగా చెప్తారు. Topic 5 లో deep.)

గుర్తుంచుకో: DDL = **structure** (ఇల్లు కట్టడం), DML = **data** (ఇంట్లో వస్తువులు పెట్టడం), DCL = **access** (ఎవరికి తాళం చెవి), TCL = **safety** (undo/save).

Real-life Scenario

> **SQL commands = ఇల్లు కట్టి, నివసించడం.**
>
> - **DDL** = ఇంటి నిర్మాణం — గోడలు, గదులు కట్టడం (CREATE), ఒక గది add/remove చేయడం (ALTER), ఇల్లు కూల్చడం (DROP). నిర్మాణం మార్పు.
> - **DML** = ఇంట్లో దైనందిన జీవితం — వస్తువులు తేవడం (INSERT), సర్దడం (UPDATE), పారేయడం (DELETE). Data మార్పు.
> - **DCL** = ఎవరికి ఏ గదికి తాళం చెవి ఇవ్వాలి (GRANT), తీసుకోవాలి (REVOKE). Access control.
> - **TCL** = "ఈ మార్పు final" (COMMIT) లేదా "అయ్యో పొరపాటు, undo చెయ్" (ROLLBACK). Safety net.

DDL — structure define చేయడం

```
-- CREATE: కొత్త table
CREATE TABLE customers (
    customer_id  INT PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(100) UNIQUE,
    city         VARCHAR(50),
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ALTER: ఉన్న table ని మార్చడం
ALTER TABLE customers ADD COLUMN phone VARCHAR(15);          -- column add
ALTER TABLE customers ALTER COLUMN city SET DEFAULT 'NA';    -- default మార్పు (Postgres)
ALTER TABLE customers DROP COLUMN phone;                     -- column తీసేయడం
ALTER TABLE customers RENAME COLUMN name TO full_name;       -- rename

-- DROP: మొత్తం table తీసేయడం (structure + data అంతా పోతుంది)
DROP TABLE customers;

-- TRUNCATE: data మొత్తం తీసేయి కానీ structure ఉంచు (fast, no WHERE)
TRUNCATE TABLE customers;
```

DROP vs TRUNCATE vs DELETE — interview classic:

అంశం	DELETE	TRUNCATE	DROP
రకం	DML	DDL	DDL
ఏం పోతుంది	ఎంచుకున్న rows (WHERE)	అన్ని rows	rows + table structure
WHERE వాడొచ్చా	అవును	కాదు	కాదు
Rollback అవుతుందా	అవును (transaction లో)	సాధారణంగా కాదు (auto-commit)	కాదు
వేగం	నెమ్మది (row-by-row, log)	చాలా వేగం (deallocate pages)	వేగం
Triggers fire అవుతాయా	అవును	కాదు	కాదు
Identity/auto-increment	reset అవ్వదు	reset అవుతుంది	—

DML — data మార్చడం

మన schema కి కొన్ని rows పెడదాం:

```
-- INSERT: కొత్త rows
INSERT INTO customers (customer_id, name, email, city) VALUES
    (1, 'Ravi',   'ravi@mail.com',   'Bangalore'),
    (2, 'Sita',   'sita@mail.com',   'Hyderabad'),
    (3, 'Arjun',  'arjun@mail.com',  'Bangalore'),
    (4, 'Meena',  'meena@mail.com',  'Chennai');

-- UPDATE: ఉన్న rows మార్చడం (WHERE లేకపోతే అన్ని rows మారతాయి — ప్రమాదం!)
UPDATE customers SET city = 'Mumbai' WHERE customer_id = 2;

-- DELETE: rows తీసేయడం
DELETE FROM customers WHERE customer_id = 4;
```

> ⚠️ **అత్యంత సాధారణ, ఖరీదైన తప్పు:** UPDATE / DELETE లో **WHERE మర్చిపోవడం.** `DELETE FROM customers;` — WHERE లేదు → మొత్తం table ఖాళీ! ఎప్పుడూ ముందు `SELECT ... WHERE ...` రాసి, correct rows వస్తున్నాయని confirm చేసి, తర్వాత WHERE ని UPDATE/DELETE కి copy చేయి. Production లో transaction లో run చేసి, verify చేసి, తర్వాత COMMIT.

DCL & TCL

```
-- DCL: permissions
GRANT SELECT, INSERT ON customers TO analyst_user;  -- చదవడం, add చేయడం అనుమతి
REVOKE INSERT ON customers FROM analyst_user;        -- add అనుమతి తీసేయడం

-- TCL: transactions (Topic 9 లో deep)
BEGIN;                                    -- transaction మొదలు
UPDATE customers SET city = 'Delhi' WHERE customer_id = 1;
SAVEPOINT before_next;                    -- checkpoint
DELETE FROM customers WHERE customer_id = 3;
ROLLBACK TO before_next;                  -- ఆ delete ని undo (update ఉంటుంది)
COMMIT;                                    -- update ని permanent చేయి
```

SQL Data Types — ముఖ్యమైనవి

సరైన type ఎంచుకోవడం performance + integrity కి కీలకం. (Type లు RDBMS ని బట్టి కొంచెం మారతాయి; ఇవి common.)

రకం	Types	ఎప్పుడు వాడాలి
Integer	`INT`, `SMALLINT`, `BIGINT`	ids, counts, quantities
Decimal (exact)	`DECIMAL(p,s)` / `NUMERIC`	**డబ్బు!** (₹123.45) — precision loss ఉండదు
Float (approx)	`FLOAT`, `REAL`, `DOUBLE`	scientific, precision అవసరం లేని చోట (డబ్బుకి వద్దు)
String	`CHAR(n)` (fixed), `VARCHAR(n)` (variable), `TEXT` (large)	names, emails, descriptions
Date/Time	`DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ`	created_at, dob (timezone కి TIMESTAMPTZ)
Boolean	`BOOLEAN`	is_active, is_deleted
Others	`UUID`, `JSON` / `JSONB`, `ENUM`, `ARRAY`	modern needs (JSONB = Postgres లో Mongo-లాంటి flexibility)

> **డబ్బుకి FLOAT వాడొద్దు!** `FLOAT` binary approximation — `0.1 + 0.2 ≠ 0.3` అవుతుంది. Money/financial కి ఎప్పుడూ `DECIMAL(10,2)` వాడు. ఇది interview + production రెండింటిలో ముఖ్యమైన gotcha.

CHAR vs VARCHAR: `CHAR(10)` ఎప్పుడూ 10 chars space తీసుకుంటుంది (padding). `VARCHAR(10)` actual length మాత్రమే (+ small overhead). Fixed-length data (state code 'KA', country 'IN') కి CHAR; variable (names) కి VARCHAR.

Mongo తో pole: Mongo schema-less — types document కి document వేరుగా ఉండొచ్చు, migration అవసరం లేదు. SQL లో schema fixed, type మార్చాలంటే `ALTER TABLE` (పెద్ద tables లో costly). Postgres `JSONB` column ఈ gap ని కొంత పూరిస్తుంది — structured columns + flexible JSON కలిపి వాడొచ్చు.

Key Points

- **4 categories:** DDL (structure: CREATE/ALTER/DROP/TRUNCATE), DML (data: INSERT/UPDATE/DELETE), DCL (access: GRANT/REVOKE), TCL (transactions: COMMIT/ROLLBACK/SAVEPOINT).
- **DELETE (DML, WHERE, rollback-able, slow) vs TRUNCATE (DDL, all rows, fast, resets identity) vs DROP (table పోతుంది).**
- **UPDATE/DELETE లో WHERE మర్చిపోవద్దు** — మొత్తం table పోతుంది. ముందు SELECT తో verify.
- **డబ్బుకి DECIMAL, FLOAT కాదు** (approximation errors). CHAR (fixed) vs VARCHAR (variable).
- Postgres `JSONB` = SQL లో Mongo-లాంటి flexibility.

Interview దృష్టి

Q: DELETE, TRUNCATE, DROP తేడా? A: **DELETE** = DML, rows ని WHERE తో selective గా తీస్తుంది, transaction లో rollback అవుతుంది, triggers fire అవుతాయి, నెమ్మది (row-by-row log). **TRUNCATE** = DDL, అన్ని rows ఒకేసారి fast (pages deallocate), WHERE లేదు, identity reset అవుతుంది, సాధారణంగా rollback కాదు. **DROP** = DDL, table structure + data మొత్తం పోతుంది.

Q: డబ్బు store చేయడానికి ఏ data type? A: `DECIMAL(p,s)` / `NUMERIC` — exact precision. `FLOAT / DOUBLE` binary floating-point approximations, rounding errors వస్తాయి (0.1+0.2 problem), financial calculations లో ప్రమాదకరం. ఉదా: `price DECIMAL(10,2)` = గరిష్ఠంగా 10 digits, 2 decimal places.

Q: CHAR vs VARCHAR ఎప్పుడు ఏది? A: `CHAR(n)` fixed-length — ఎప్పుడూ n bytes (padding తో), fixed-size data (country code, Y/N flags) కి, minor speed edge. `VARCHAR(n)` variable-length — actual data + length overhead, variable data (names, emails) కి, space-efficient. చాలావరకు VARCHAR default choice.

5. Querying (SELECT & filtering)

వివరణ

`SELECT` = database నుండి data ని **చదవడం (read).** ఇది SQL లో అత్యధికంగా వాడే command. దీని full anatomy:

```
SELECT   column1, column2        -- ఏ columns కావాలి
FROM     table_name              -- ఏ table నుండి
WHERE    condition               -- ఏ rows (filter)
GROUP BY column                  -- grouping (Topic 7)
HAVING   group_condition         -- group filter (Topic 7)
ORDER BY column [ASC|DESC]       -- sort
LIMIT    n OFFSET m;             -- ఎన్ని rows
```

అత్యంత ముఖ్యం — SQL logical execution order. నువ్వు రాసే order కి, database process చేసే order కి **తేడా** ఉంది. ఇది interview లో అడుగుతారు, మరియు ఇది తెలిస్తే చాలా confusion clear అవుతుంది (ఉదా: WHERE లో alias ఎందుకు వాడలేం):

```
రాసే order:     SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
process order:   FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
                 (1)     (2)      (3)        (4)      (5)       (6)        (7)
```

అంటే **SELECT (aliases) WHERE కంటే తర్వాత evaluate అవుతుంది** — అందుకే WHERE లో SELECT alias వాడలేం, కానీ ORDER BY లో వాడొచ్చు (అది SELECT తర్వాత).

Real-life Scenario

> **SELECT query = Amazon search filters.**
>
> నువ్వు Amazon లో వెతుకుతున్నావు:
>
> - **FROM products** — ఏ catalog నుండి.
> - **WHERE price < 1000 AND brand = 'Boat'** — filters (left sidebar).
> - **ORDER BY rating DESC** — "Sort by: Customer rating".
> - **LIMIT 20** — మొదటి page లో 20 results.
> - **SELECT name, price, rating** — ప్రతి product కి ఏ details చూపించాలి.
>
> అంటే SQL query అంటే ఏమీ కొత్త కాదు — నువ్వు రోజూ చేసే "filter + sort + show" ని database భాషలో రాయడమే.

మన dataset (setup — ఒకసారి run చేయి)

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(100),
    email       VARCHAR(100),
    city        VARCHAR(50),
    age         INT
);
INSERT INTO customers VALUES
    (1,'Ravi','ravi@mail.com','Bangalore',28),
    (2,'Sita','sita@mail.com','Hyderabad',34),
    (3,'Arjun','arjun@gmail.com','Bangalore',22),
    (4,'Meena','meena@mail.com','Chennai',NULL),
    (5,'Kiran','kiran@gmail.com','Hyderabad',45);
```

SELECT, WHERE — basics

```
SELECT * FROM customers;                              -- అన్ని columns, అన్ని rows
SELECT name, city FROM customers;                     -- కొన్ని columns మాత్రమే
SELECT name, city FROM customers WHERE city = 'Bangalore';    -- filter

-- column alias (AS) — output లో పేరు మార్చడం
SELECT name AS customer_name, age AS years FROM customers;
```

Operators — WHERE లో వాడేవి

Operator	అర్థం	ఉదాహరణ
`=` `<>` `!=`	సమానం, సమానం కాదు	`WHERE city = 'Chennai'`
`<` `>` `<=` `>=`	పోలిక	`WHERE age > 30`
`AND` `OR` `NOT`	logical combine	`WHERE age > 25 AND city = 'Bangalore'`
`BETWEEN a AND b`	range (inclusive, రెండు ends కలిపి)	`WHERE age BETWEEN 25 AND 40`
`IN (...)`	list లో ఏదో ఒకటి	`WHERE city IN ('Bangalore','Chennai')`
`LIKE`	pattern match	`WHERE email LIKE '%gmail.com'`
`IS NULL / IS NOT NULL`	NULL check	`WHERE age IS NULL`

```
SELECT * FROM customers WHERE age BETWEEN 25 AND 40;    -- 25..40 (both inclusive)
SELECT * FROM customers WHERE city IN ('Bangalore','Chennai');
SELECT * FROM customers WHERE age > 25 AND city = 'Hyderabad';
SELECT * FROM customers WHERE NOT city = 'Bangalore';
```

NULL — SQL లో అత్యంత గమ్మత్తైన అంశం (interview favorite)

NULL = "value తెలియదు / లేదు" (unknown/missing), 0 కాదు, empty string కాదు. NULL తో పోలికలు **UNKNOWN** ఇస్తాయి, TRUE/FALSE కాదు — అందుకే:

```
SELECT * FROM customers WHERE age = NULL;      -- ❌ తప్పు! ఏ row రాదు (NULL=NULL → unknown)
SELECT * FROM customers WHERE age IS NULL;     -- ✅ సరైనది → Meena వస్తుంది
SELECT * FROM customers WHERE age <> 30;       -- ⚠️ NULL age (Meena) రాదు! (unknown)
```

కీలక నియమం: NULL తో `=`, `<>`, `<`, `>` ఏదీ పనిచేయదు — ఎప్పుడూ `IS NULL` / `IS NOT NULL` వాడు. అలాగే `age <> 30` NULL rows ని **వదిలేస్తుంది** — ఇది చాలా bugs కి కారణం.

```
-- NULL ని handle చేయడం: COALESCE (మొదటి non-NULL value ఇస్తుంది)
SELECT name, COALESCE(age, 0) AS age_or_zero FROM customers;   -- Meena → 0
```

DISTINCT — duplicates తీసేయడం

```
SELECT DISTINCT city FROM customers;   -- Bangalore, Hyderabad, Chennai (unique cities)
SELECT DISTINCT city, age FROM customers;  -- (city,age) combination unique
```

LIKE — pattern matching

Wildcards: `%` = ఎన్ని characters అయినా (0+), `_` = సరిగ్గా ఒక్క character.

```
SELECT * FROM customers WHERE email LIKE '%gmail.com';  -- gmail తో ముగిసేవి
SELECT * FROM customers WHERE name  LIKE 'A%';          -- A తో మొదలయ్యేవి → Arjun
SELECT * FROM customers WHERE name  LIKE '_i%';         -- 2వ letter 'i' → Sita, Kiran
SELECT * FROM customers WHERE name  LIKE '%a%';         -- ఎక్కడైనా 'a'
-- case-insensitive: Postgres లో ILIKE
SELECT * FROM customers WHERE name ILIKE 'r%';          -- Ravi (case ignore)
```

> **Performance gotcha:** `LIKE '%text'` (ముందు %) index వాడలేదు → full table scan (slow). `LIKE 'text%'` (చివర %) index వాడగలదు (fast). ఇది Topic 12 (indexing) లో ముఖ్యం.

ORDER BY — sorting

```
SELECT name, age FROM customers ORDER BY age;                 -- ascending (default)
SELECT name, age FROM customers ORDER BY age DESC;            -- descending
SELECT * FROM customers ORDER BY city ASC, age DESC;          -- city ఆరోహణ, ఒకే city లో age
అవరోహణ
-- NULL sorting: Postgres లో NULLS FIRST / NULLS LAST
SELECT name, age FROM customers ORDER BY age DESC NULLS LAST;
```

LIMIT & OFFSET — pagination

```
SELECT * FROM customers ORDER BY customer_id LIMIT 3;          -- మొదటి 3
SELECT * FROM customers ORDER BY customer_id LIMIT 3 OFFSET 3;    -- 4-6వ (page 2)
-- MySQL/Postgres: LIMIT n OFFSET m
-- SQL Server/Oracle: OFFSET m ROWS FETCH NEXT n ROWS ONLY
```

> **Pagination gotcha:** `OFFSET` పెద్దదైతే (ఉదా: OFFSET 1000000) DB మొదటి 10 lakh rows ని skip చేయడానికి scan చేస్తుంది → slow. Large datasets కి **keyset/cursor pagination** (`WHERE id > last_seen_id LIMIT n`) వాడతారు — ఇది SSE interview లో scaling question.

పూర్తి query — అన్నీ కలిపి

```
-- Bangalore/Hyderabad లో, age 25+ ఉన్న customers, age ఎక్కువ నుండి, top 2
SELECT name, city, age
FROM   customers
WHERE  city IN ('Bangalore','Hyderabad') AND age >= 25
ORDER BY age DESC
LIMIT  2;
-- → Kiran(45,Hyderabad), Sita(34,Hyderabad)
```

Key Points

- **Logical execution order:** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. అందుకే WHERE లో SELECT alias వాడలేం (SELECT తర్వాత), ORDER BY లో వాడొచ్చు.
- **NULL ≠ 0 ≠ ''.** `= NULL` పనిచేయదు → `IS NULL` వాడు. `<> value` NULL rows ని వదిలేస్తుంది. COALESCE తో handle.
- **BETWEEN inclusive** (రెండు ends). `IN` = list membership. `DISTINCT` = unique.
- **LIKE:** `%` (0+ chars), `_` (1 char). `LIKE '%x'` index వాడదు (slow), `LIKE 'x%'` వాడగలదు.
- **ORDER BY** multiple columns + ASC/DESC. **LIMIT/OFFSET** pagination — large OFFSET slow, keyset pagination వాడు.

Interview దృష్టి

Q: SQL query లో clauses execute అయ్యే order ఏమిటి? A: రాసేది `SELECT...FROM...WHERE...GROUP BY...HAVING...ORDER BY...LIMIT` కానీ **execute అయ్యేది** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. అందుకే SELECT లో define చేసిన alias ని WHERE లో వాడలేం (WHERE ముందు run అవుతుంది), కానీ ORDER BY లో వాడొచ్చు.

Q: `WHERE age = NULL` ఎందుకు పనిచేయదు? A: NULL అంటే "unknown". SQL లో ఏదైనా NULL తో పోలిక (=, <>, <, >) **UNKNOWN** ఇస్తుంది, TRUE కాదు → ఆ row filter లో pass అవ్వదు. NULL check కి `IS NULL` / `IS NOT NULL` మాత్రమే వాడాలి. అలాగే `age <> 30` కూడా NULL age rows ని silently వదిలేస్తుంది — common bug.

Q: `LIKE '%abc'` slow ఎందుకు? A: Leading wildcard (`%` ముందు) ఉంటే B-tree index prefix మీద search చేయలేదు (index sorted by prefix), → full table scan అవుతుంది. `LIKE 'abc%'` (trailing wildcard) prefix

తెలుసు కాబట్టి index range scan చేయగలదు (fast). Full-text search అవసరమైతే dedicated full-text index / search engine (Elasticsearch) వాడాలి.

6. Joins

వివరణ

Join = రెండు (లేదా అంతకంటే ఎక్కువ) tables ని ఒక **common column** (సాధారణంగా foreign key) ఆధారంగా **కలిపి** ఒకే result గా చూపించడం. ఇది relational databases యొక్క **అసలు శక్తి** — data ని normalized గా (redundancy లేకుండా) separate tables లో store చేసి, అవసరమైనప్పుడు join తో కలుపుతాం.

ఎందుకు అవసరం? `orders` table లో customer_id ఉంటుంది కానీ customer name ఉండదు (redundancy తప్పించడానికి). Order తో పాటు customer name కావాలంటే — `orders` ని `customers` తో join చేయాలి.

Mongo తో pole: Mongo లో joins బలహీనం (అందుకే data ని embed చేస్తారు — order document లోపలే customer info పెడతారు). `$lookup` aggregation stage ఒక join లాంటిది కానీ SQL joins అంత powerful/optimized కాదు. **ఇదే SQL vs NoSQL లో ముఖ్యమైన తేడా** — relational DB joins కి designed, document DB embedding కి designed.

Real-life Scenario

> **Join = రెండు registers ని match చేయడం.**
>
> ఒక school లో రెండు registers ఉన్నాయి:
>
> - **Register A (students):** roll_no, పేరు.
> - **Register B (marks):** roll_no, subject, marks.
>
> "ప్రతి student పేరుతో పాటు వాళ్ళ marks కావాలి" అంటే — రెండు registers ని **roll_no** ఆధారంగా పక్కపక్కన పెట్టి match చేస్తావు. అదే **join.**
>
> - Register A లో ఉన్న ప్రతి student కి B లో marks ఉంటేనే తీసుకుంటే → **INNER JOIN.**
> - A లో ఉన్న అందరు students ని ఉంచి, marks లేకపోతే ఖాళీ (NULL) → **LEFT JOIN.**
>
> అంటే join అంటే "ఏ register ని base గా తీసుకోవాలి, match లేని వాళ్ళని ఏం చేయాలి" అనే decision.

మన running example (setup)

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(50),
    city        VARCHAR(50)
);
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT,       -- FK → customers (కానీ కొన్ని orders కి invalid/no customer)
    amount      DECIMAL(10,2)
);
INSERT INTO customers VALUES
```

```
    (1,'Ravi','Bangalore'), (2,'Sita','Hyderabad'),
    (3,'Arjun','Chennai'),  (4,'Meena','Delhi');       -- Meena కి orders లేవు
INSERT INTO orders VALUES
    (101, 1, 500), (102, 1, 300),    -- Ravi కి 2 orders
    (103, 2, 900),                   -- Sita కి 1 order
    (104, 5, 250);                   -- customer 5 → customers లో లేదు (orphan)
-- Arjun(3): customer ఉన్నాడు కానీ order లేదు. customer 5: order ఉంది కానీ customer లేదు.
```

JOIN రకాలు — Venn diagrams తో

```
INNER JOIN          LEFT JOIN           RIGHT JOIN          FULL OUTER JOIN
   A   B               A   B               A   B               A   B

  ┌──┐┌──┐            ┌──┐┌──┐            ┌──┐┌──┐            ┌──┐┌──┐
  │ ▓▓▓▓ │            │▓▓▓▓▓ │            │ ▓▓▓▓▓│            │▓▓▓▓▓▓│
  └──┘└──┘            └──┘└──┘            └──┘└──┘            └──┘└──┘

  match మాత్రమే         A అంతా + match        B అంతా + match        A,B రెండూ అంతా
```

1. INNER JOIN — రెండు tables లోనూ match ఉన్నవి మాత్రమే

```
SELECT c.name, o.order_id, o.amount
FROM   customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

name	order_id	amount
Ravi	101	500
Ravi	102	300
Sita	103	900

Arjun (order లేదు), Meena (order లేదు), order 104 (customer లేదు) — ఏవీ రావు. INNER = intersection మాత్రమే. ఇది అత్యధికంగా వాడే join.

2. LEFT JOIN (LEFT OUTER JOIN) — ఎడమ table అంతా + కుడి match

```
SELECT c.name, o.order_id, o.amount
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

name	order_id	amount
Ravi	101	500
Ravi	102	300
Sita	103	900
Arjun	NULL	NULL
Meena	NULL	NULL

అన్ని customers వస్తారు. Order లేని Arjun/Meena కి order columns NULL. order 104 (customer లేని) రాదు — అది left (customers) లో లేదు. **అత్యంత useful pattern:** "orders లేని customers ఎవరు?" → `WHERE o.order_id IS NULL` .

```
-- ఏ customer ఏ order ఇవ్వలేదు?
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;          -- → Arjun, Meena
```

3. RIGHT JOIN (RIGHT OUTER JOIN) — కుడి table అంతా + ఎడమ match

```
SELECT c.name, o.order_id, o.amount
FROM   customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

name	order_id	amount
Ravi	101	500
Ravi	102	300
Sita	103	900
NULL	104	250

అన్ని orders వస్తాయి. order 104 కి customer లేడు → name NULL. (Arjun/Meena రారు — వాళ్ళకి orders లేవు.)

> **Tip:** RIGHT JOIN అరుదుగా వాడతారు — ఎందుకంటే `A RIGHT JOIN B = B LEFT JOIN A` . చాలామంది tables order మార్చి LEFT JOIN వాడతారు (చదవడానికి సులభం). "ఎప్పుడూ LEFT JOIN వాడు" అనేది common team convention.

4. FULL OUTER JOIN — రెండు tables అంతా

```
SELECT c.name, o.order_id, o.amount
FROM   customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

name	order_id	amount
Ravi	101	500
Ravi	102	300
Sita	103	900
Arjun	NULL	NULL
Meena	NULL	NULL
NULL	104	250

అన్నీ వస్తాయి — matched + unmatched customers + unmatched orders. Match లేని చోట NULL. (MySQL లో FULL OUTER JOIN direct support లేదు — `LEFT JOIN UNION RIGHT JOIN` తో చేయాలి. Postgres, SQL Server, Oracle support చేస్తాయి.)

5. CROSS JOIN — Cartesian product (ప్రతి row × ప్రతి row)

```
SELECT c.name, o.order_id FROM customers c CROSS JOIN orders o;
-- 4 customers × 4 orders = 16 rows (ప్రతి customer ప్రతి order తో pair)
```

Condition ఉండదు. అరుదుగా వాడతారు — ఉదా: ప్రతి product కి ప్రతి size combination generate చేయడానికి. ⚠️ **ప్రమాదం:** పొరపాటున ON మర్చిపోతే (`FROM a, b` without WHERE) accidental cross join వస్తుంది → millions of rows, DB hang.

6. SELF JOIN — ఒక table ని అదే table తో join

ఒక table లోని rows ని అదే table లోని ఇతర rows తో పోల్చడానికి. ఉదా: employees table లో ప్రతి employee, వాళ్ళ manager (అతను కూడా ఒక employee) పేరు.

```
CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(50),
    manager_id INT          -- ఇది కూడా emp_id ని reference చేస్తుంది
);
INSERT INTO employees VALUES
    (1,'Ravi',NULL),    -- Ravi = CEO (manager లేడు)
    (2,'Sita',1),       -- Sita's manager = Ravi
    (3,'Arjun',1),      -- Arjun's manager = Ravi
    (4,'Kiran',2);      -- Kiran's manager = Sita

-- ప్రతి employee + వాళ్ళ manager పేరు
SELECT e.name AS employee, m.name AS manager
FROM   employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

employee	manager
Ravi	NULL
Sita	Ravi
Arjun	Ravi
Kiran	Sita

కీలకం: అదే table ని **రెండు వేర్వేరు aliases** (`e` , `m`) తో వాడతాం — ఒకటి employee గా, ఒకటి manager గా. Self join లేకపోతే ఇది impossible.

JOIN vs subquery — ఏది వాడాలి?

రెండూ తరచుగా ఒకే result ఇస్తాయి. JOIN సాధారణంగా **వేగం** (optimizer better handle చేస్తుంది), subquery కొన్నిసార్లు **చదవడానికి సులభం**. "Bangalore customers ఇచ్చిన orders":

```
-- JOIN version (సాధారణంగా faster)
SELECT o.* FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.city = 'Bangalore';

-- Subquery version (Topic 7 లో deep)
SELECT * FROM orders
WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'Bangalore');
```

Key Points

- **Join = tables ని common column ఆధారంగా కలపడం** (relational DB యొక్క core శక్తి). Mongo embedding వాడుతుంది; joins బలహీనం.
- **INNER** = match మాత్రమే (intersection). **LEFT** = ఎడమ అంతా + కుడి match (unmatched → NULL). **RIGHT** = దానికి reverse. **FULL** = రెండూ అంతా.
- `LEFT JOIN ... WHERE right.id IS NULL` = "match లేని left rows" (అత్యంత useful pattern — orphans, missing).
- **CROSS JOIN** = Cartesian product (అన్ని combinations, ప్రమాదకరం accidental గా). **SELF JOIN** = table ని అదే table తో (manager-employee, hierarchy).
- Table aliases (`c` , `o`) readability కి; self join కి తప్పనిసరి.

Interview దృష్టి

Q: INNER JOIN vs LEFT JOIN తేడా? A: **INNER JOIN** రెండు tables లోనూ match ఉన్న rows మాత్రమే ఇస్తుంది. **LEFT JOIN** ఎడమ table లోని అన్ని rows ఇస్తుంది — కుడి table లో match లేకపోతే ఆ columns NULL. ఉదా: customers LEFT JOIN orders → order లేని customers కూడా వస్తారు (NULL orders తో), INNER JOIN అయితే రారు.

Q: "orders ఇవ్వని customers" ని ఎలా కనుక్కుంటావు? A: `customers LEFT JOIN orders ON ... WHERE orders.id IS NULL` . LEFT JOIN అన్ని customers ని తెస్తుంది; order లేని వాళ్ళకి order columns NULL అవుతాయి; ఆ

NULL ని filter చేస్తే match లేని (order ఇవ్వని) customers వస్తారు. ఇది "anti-join" pattern.

Q: Self join అంటే ఏమిటి, ఎప్పుడు వాడతావు? A: ఒక table ని అదే table తో join చేయడం (రెండు వేర్వేరు aliases తో). Hierarchical/recursive data కి — employee-manager (ఇద్దరూ employees table లో), category-parent category, friend-of-friend. ఉదా: `employees e JOIN employees m ON e.manager_id = m.emp_id` → ప్రతి employee పక్కన వాళ్ళ manager పేరు.

Q: RIGHT JOIN అవసరమా? A: Technically అవసరం లేదు — ఏ RIGHT JOIN అయినా tables order మార్చి LEFT JOIN గా రాయవచ్చు (`A RIGHT JOIN B = B LEFT JOIN A`). చాలా teams readability కోసం "ఎప్పుడూ LEFT JOIN" convention పాటిస్తాయి. కానీ RIGHT JOIN existence తెలిసి ఉండాలి.

7. Aggregation & Advanced (GROUP BY, subqueries, CTEs, window functions)

వివరణ

ఇప్పటివరకు individual rows చూశాం. కానీ నిజ ప్రపంచ ప్రశ్నలు తరచుగా **summary** అడుగుతాయి: "ఏ city లో ఎంతమంది customers?", "ప్రతి customer మొత్తం ఎంత spend చేశాడు?", "top 3 spenders ఎవరు?". దీనికి **aggregation** (గుంపుగా చేసి లెక్కించడం) వాడతాం.

Aggregate functions = చాలా rows ని తీసుకుని ఒక్క value ఇస్తాయి:

Function	ఏం చేస్తుంది	NULL treatment
`COUNT(*)`	rows సంఖ్య (NULL కూడా లెక్క)	అన్నీ లెక్క
`COUNT(col)`	ఆ column లో non-NULL values	NULL skip
`SUM(col)`	మొత్తం	NULL skip
`AVG(col)`	సగటు	NULL skip (denominator లో కూడా!)
`MIN(col) / MAX(col)`	కనిష్ఠ / గరిష్ఠ	NULL skip

Real-life Scenario

> **GROUP BY = బట్టలను రంగుల వారీగా విడగొట్టడం.**
>
> నీ దగ్గర 100 బట్టలు ఉన్నాయి. "ఏ రంగు బట్టలు ఎన్ని?" అని తెలియాలంటే — ముందు అన్నిటినీ **రంగుల వారీగా గుట్టలు (groups)** చేస్తావు: ఎరుపు గుట్ట, నీలం గుట్ట... తర్వాత ప్రతి గుట్టలో ఎన్ని ఉన్నాయో **లెక్కిస్తావు (COUNT)**.
>
> - **GROUP BY color** = రంగుల వారీగా గుట్టలు చేయడం.
> - **COUNT(*)** = ప్రతి గుట్టలో ఎన్ని.
> - **HAVING count > 10** = 10 కంటే ఎక్కువ ఉన్న రంగుల గుట్టలు మాత్రమే చూపించడం.
>
> అంటే aggregation అంటే "విడగొట్టి → లెక్కించి → filter చేయడం." రోజువారీ పని.

GROUP BY — data ని groups గా చేయడం

```sql
-- dataset: orders(order_id, customer_id, city, amount)
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT,
    city        VARCHAR(50),
    amount      DECIMAL(10,2)
);
INSERT INTO orders VALUES
    (1,10,'Bangalore',500), (2,10,'Bangalore',300),
    (3,20,'Hyderabad',900), (4,30,'Bangalore',150),
    (5,20,'Hyderabad',600), (6,40,'Chennai',200);

-- ప్రతి city లో ఎన్ని orders, మొత్తం amount, సగటు
SELECT city,
       COUNT(*)     AS num_orders,
       SUM(amount)  AS total,
       AVG(amount)  AS avg_amount
FROM   orders
GROUP BY city;
```

city	num_orders	total	avg_amount
Bangalore	3	950	316.67
Hyderabad	2	1500	750.00
Chennai	1	200	200.00

కీలక నియమం (interview trap): GROUP BY వాడినప్పుడు, SELECT లో వచ్చే ప్రతి column **ఒకటో GROUP BY లో ఉండాలి, లేదా aggregate function లో ఉండాలి.** లేకపోతే error (ఏ value చూపించాలో DB కి తెలియదు).

```sql
SELECT city, amount FROM orders GROUP BY city;   -- ❌ error! amount ఏ value? (group లో చాలా ఉన్నాయి)
SELECT city, SUM(amount) FROM orders GROUP BY city;  -- ✅ SUM aggregate
```

HAVING — groups ని filter చేయడం (WHERE vs HAVING)

WHERE = rows ని filter చేస్తుంది (grouping ముందు). HAVING = groups ని filter చేస్తుంది (grouping తర్వాత). ఇది అత్యంత important interview distinction.

```sql
-- 800+ మొత్తం amount ఉన్న cities మాత్రమే
SELECT city, SUM(amount) AS total
FROM   orders
WHERE  amount > 100          -- ముందు: 100 దాటిన individual orders మాత్రమే (rows filter)
GROUP BY city
HAVING SUM(amount) > 800;    -- తర్వాత: group total 800 దాటినవి (groups filter)
```

Aspect	WHERE	HAVING
ఎప్పుడు apply	GROUP BY **ముందు**	GROUP BY **తర్వాత**
దేని మీద	individual rows	grouped results
Aggregate వాడొచ్చా	❌ కాదు (`WHERE SUM()` invalid)	✅ అవును (`HAVING SUM() > x`)
ఉదాహరణ	`WHERE amount > 100`	`HAVING COUNT(*) > 5`

> **Performance tip:** వీలైనంత filtering ని WHERE లో చేయి (rows తగ్గించి తర్వాత group). HAVING ని groups మీద మాత్రమే వాడు. `WHERE amount > 100 ... HAVING SUM(amount) > 800` — రెండూ వేర్వేరు పనులు.

Subqueries (nested queries) — query లోపల query

Subquery = మరో query లోపల ఉన్న SELECT. 3 రకాలు:

1. Scalar subquery — ఒక్క value ఇస్తుంది:

```
-- సగటు కంటే ఎక్కువ amount ఉన్న orders
SELECT * FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);   -- inner → ఒక్క number (441.67)
```

2. Multi-row subquery — list ఇస్తుంది (IN, ANY, ALL తో):

```
-- Bangalore/Hyderabad orders ఇచ్చిన customers ఇచ్చిన అన్ని orders
SELECT * FROM orders
WHERE customer_id IN (
    SELECT DISTINCT customer_id FROM orders WHERE city IN ('Bangalore','Hyderabad')
);
```

3. Correlated subquery — outer query లోని ప్రతి row కి inner query run అవుతుంది:

```
-- ప్రతి order, అదే city సగటు కంటే ఎక్కువా?
SELECT o1.* FROM orders o1
WHERE o1.amount > (
    SELECT AVG(o2.amount) FROM orders o2 WHERE o2.city = o1.city   -- o1 ని reference
(correlated)
);
```

> **Correlated subquery gotcha:** ఇది outer row కి ఒకసారి run అవుతుంది → పెద్ద tables లో **నెమ్మది** (N queries). తరచుగా దీన్ని **JOIN లేదా window function** గా rewrite చేస్తే వేగం. Interview లో "ఇది optimize చెయ్" అంటే ఇదే.

CTE (Common Table Expression) — `WITH` clause

CTE = ఒక temporary named result set — query ని **చదవగలిగేలా, భాగాలుగా** విడగొట్టడానికి. Nested subqueries కంటే చాలా clean.

```
-- ప్రతి customer total, తర్వాత avg కంటే ఎక్కువ spend చేసిన వాళ్ళు
WITH customer_totals AS (
    SELECT customer_id, SUM(amount) AS total
    FROM   orders
    GROUP BY customer_id
)
SELECT * FROM customer_totals
WHERE total > (SELECT AVG(total) FROM customer_totals);
```

CTE benefits: (1) readability (top-down చదవొచ్చు), (2) ఒకే subquery ని multiple చోట్ల reuse, (3) **recursive CTEs** (hierarchies — org chart, category tree traverse చేయడానికి). Complex queries లో subqueries కంటే CTE preferred.

```
-- Recursive CTE: manager hierarchy (Ravi కింద ఎవరెవరు)
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS level
    FROM   employees WHERE emp_id = 1              -- anchor: top boss
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.level + 1
    FROM   employees e JOIN org o ON e.manager_id = o.emp_id    -- recursive part
)
SELECT * FROM org;
```

Window Functions — aggregation without collapsing rows (SSE ముఖ్యం)

సాధారణ GROUP BY rows ని **collapse** చేస్తుంది (ప్రతి group → 1 row). **Window function** aggregate compute చేస్తుంది కానీ **అన్ని rows ఉంచుతుంది** — ప్రతి row పక్కన aggregate value చూపిస్తుంది. ఇది SSE-level SQL లో అత్యంత powerful feature.

Syntax: `function() OVER (PARTITION BY ... ORDER BY ...)`.

```
-- ప్రతి order పక్కన, ఆ city మొత్తం (rows collapse అవ్వకుండా)
SELECT order_id, city, amount,
       SUM(amount) OVER (PARTITION BY city)         AS city_total,
       AVG(amount) OVER (PARTITION BY city)         AS city_avg
FROM   orders;
-- ప్రతి order row అలాగే ఉంటుంది, పక్కన దాని city total/avg వస్తుంది
```

Ranking functions — top-N per group కి:

Function	ఏం చేస్తుంది	ties (సమానం) కి
`ROW_NUMBER()`	1,2,3,4... (unique)	ties కి కూడా వేర్వేరు numbers
`RANK()`	1,2,2,4...	ties కి same rank, తర్వాత **gap** (3 skip)
`DENSE_RANK()`	1,2,2,3...	ties కి same rank, **gap లేదు**

```
-- ప్రతి city లో top spender (rank 1)
SELECT * FROM (
    SELECT order_id, city, amount,
           ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount DESC) AS rn
    FROM orders
) ranked
WHERE rn = 1;    -- ప్రతి city యొక్క highest order
```

```
-- LAG/LEAD: ముందు/తర్వాత row value (running comparison కి)
SELECT order_id, amount,
       LAG(amount)  OVER (ORDER BY order_id) AS prev_amount,   -- ముందు order amount
       amount - LAG(amount) OVER (ORDER BY order_id) AS diff
FROM orders;
```

> **ఎప్పుడు window function vs GROUP BY?** ప్రతి row + దాని group aggregate రెండూ కావాలంటే → **window** (running totals, ranks, "each vs average", top-N per group). Group summary మాత్రమే కావాలంటే → **GROUP BY**. Window functions interview లో "top-N per category" ప్రశ్నకి go-to.

Key Points

- **Aggregate functions** (COUNT/SUM/AVG/MIN/MAX) చాలా rows → 1 value. `COUNT(*)` NULL లెక్క, `COUNT(col)` /SUM/AVG NULL skip.
- **GROUP BY** = groups గా విడగొట్టడం. SELECT లో column లు GROUP BY లో ఉండాలి లేదా aggregate లో ఉండాలి.
- **WHERE (rows, grouping ముందు) vs HAVING (groups, grouping తర్వాత, aggregate వాడొచ్చు).**
- **Subqueries:** scalar (1 value), multi-row (IN), correlated (outer row కి run అవుతుంది — slow, JOIN గా rewrite). **CTE (WITH)** = readable, reusable, recursive.
- **Window functions** = aggregate but rows collapse అవ్వవు. `OVER(PARTITION BY ORDER BY)`. ROW_NUMBER (unique), RANK (gap), DENSE_RANK (no gap). Top-N per group కి perfect.

Interview దృష్టి

Q: WHERE vs HAVING తేడా? A: **WHERE** individual rows ని GROUP BY **ముందు** filter చేస్తుంది; aggregate functions వాడలేం. **HAVING** grouped results ని GROUP BY **తర్వాత** filter చేస్తుంది; aggregate వాడొచ్చు (`HAVING SUM(x) > 100`). ఉదా: `WHERE amount > 100` (big orders మాత్రమే), `HAVING COUNT(*) > 5` (5+ orders ఉన్న groups). Performance కి filtering ని వీలైనంత WHERE లో చేయాలి.

Q: GROUP BY vs Window function ఎప్పుడు ఏది? A: **GROUP BY** rows ని collapse చేస్తుంది — ప్రతి group కి ఒక summary row (city → total). **Window function** aggregate compute చేస్తుంది కానీ అన్ని rows ఉంచుతుంది — ప్రతి

row పక్కన group value (ప్రతి order + దాని city total). Running totals, ranks, "each vs group average", top-N-per-group లాంటివి window functions కి. Detail + summary రెండూ కావాలంటే window.

Q: ROW_NUMBER vs RANK vs DENSE_RANK? A: సమాన values (ties) వచ్చినప్పుడు తేడా. **ROW_NUMBER** ఎప్పుడూ unique (1,2,3,4 — ties కి arbitrary order). **RANK** ties కి same rank, తర్వాత gap (1,2,2,4). **DENSE_RANK** ties కి same rank, gap లేదు (1,2,2,3). "Top 3 unique salaries" → DENSE_RANK; "exactly N rows" → ROW_NUMBER.

Q: Correlated subquery slow అని ఎందుకు అంటారు? A: అది outer query లోని **ప్రతి row కి ఒకసారి** execute అవుతుంది — N rows ఉంటే N times inner query run అవుతుంది (nested loop). పెద్ద tables లో ఇది చాలా నెమ్మది. తరచుగా దీన్ని JOIN లేదా window function గా rewrite చేస్తే optimizer ఒకే pass లో చేస్తుంది → చాలా వేగం.

Part 3 — Design (డిజైన్)

> Table లు రాయడం తెలుసు; కానీ **సరైన tables** ఎలా design చేయాలి? ఒకే పెద్ద table లో అంతా పెడితే ఏం సమస్యలు వస్తాయి? దీనికి జవాబే **normalization** — data ని redundancy లేకుండా, anomalies రాకుండా organize చేసే systematic process. ఇది relational design యొక్క గుండె, interview లో తప్పకుండా వచ్చే topic.

---

8. Functional Dependencies & Normalization

వివరణ

Normalization = ఒక పెద్ద table ని, redundancy మరియు anomalies తగ్గించేలా, చిన్న tables గా systematic గా విభజించడం. E.F. Codd దీన్ని normal forms (1NF, 2NF, 3NF, BCNF...) గా నిర్వచించాడు.

ముందు అర్థం చేసుకో — ఒకే పెద్ద table లో అంతా పెడితే ఏం జరుగుతుంది? ఈ "bad" table చూడు:

```
enrollments (ప్రతిదీ ఒకే table లో — un-normalized)
+---------+----------+-----------+---------+-------------+-------------+
| stud_id | stud_name| course_id | c_name  | instructor  | instr_phone |
+---------+----------+-----------+---------+-------------+-------------+
|   1     | Ravi     |   C01     | DBMS    | Prof. Rao   | 99999       |
|   1     | Ravi     |   C02     | OS      | Prof. Devi  | 88888       |
|   2     | Sita     |   C01     | DBMS    | Prof. Rao   | 99999       |
+---------+----------+-----------+---------+-------------+-------------+
```

ఇక్కడ సమస్యలు (**anomalies** — interview లో ఇవి explain చేయాలి):

Anomaly	అర్థం	ఉదాహరణ
Insertion anomaly	ఒక data insert చేయాలంటే అనవసర data కూడా అవసరం	కొత్త course add చేయాలంటే, ఒక student enroll అయ్యేవరకు add చేయలేం (stud_id NULL అవ్వదు)
Update anomaly	ఒక fact మార్చాలంటే చాలా rows update చేయాలి	Prof. Rao phone మారితే, DBMS ఉన్న ప్రతి row update చేయాలి; ఒకటి miss అయితే inconsistency
Deletion anomaly	ఒక data delete చేస్తే వేరే అవసరమైన data కూడా పోతుంది	Ravi OS enrollment delete చేస్తే, Prof. Devi info మొత్తం పోతుంది

Redundancy (Prof. Rao, 99999 అనేక సార్లు) అనేది ఈ anomalies అన్నిటికీ మూలం. Normalization ఈ redundancy ని తీసేస్తుంది.

Real-life Scenario

> **Normalization = వస్తువులను సరైన అరల్లో పెట్టడం.**
>
> ఊహించు — నీ ఇంట్లో ప్రతి గదిలో ఒక్కో చోట "electricity bill amount" రాసి పెట్టావు (bedroom, hall, kitchen). ఇప్పుడు bill మారితే — **మూడు చోట్లా మార్చాలి.** ఒకటి మర్చిపోతే, ఏది నిజమో తెలియదు (inconsistency).
>
> బదులుగా — **ఒకే చోట** (ఒక "bills" file లో) రాసి, మిగతా చోట్ల "ఆ file చూడు" అని reference పెడితే? Bill మారితే **ఒకే చోట** మార్చు, అందరికీ correct.
>
> అదే normalization: **ప్రతి fact ని ఒకే చోట (single source of truth) store చేయి, మిగతా చోట్ల foreign key తో reference చేయి.** "One fact, one place."

Functional Dependency (FD) — normalization యొక్క ఆధారం

Functional Dependency `X → Y` = "X తెలిస్తే, Y unique గా determine అవుతుంది." X ని బట్టి Y fix అవుతుంది.

- `stud_id → stud_name` — student id తెలిస్తే name తెలుస్తుంది (ఒక id కి ఒకే name). ✅
- `course_id → course_name, instructor` — course id తెలిస్తే course details fix. ✅
- `stud_name → stud_id` — ❌ (ఇద్దరు "Ravi" ఉండొచ్చు, name నుండి id చెప్పలేం).

కీలక పదాలు:

- **Prime attribute** = ఏదైనా candidate key లో భాగమైన attribute.
- **Non-prime attribute** = ఏ candidate key లోనూ లేని attribute.
- **Partial dependency** = non-prime attribute, composite key యొక్క **భాగం** మీద ఆధారపడటం.
- **Transitive dependency** = non-prime → non-prime (A → B → C, కాబట్టి A → C).

ఈ పదాలు normal forms నిర్వచించడానికి కావాలి. భయపడకు — worked examples తో clear అవుతుంది.

1NF (First Normal Form) — atomic values

నియమం: ప్రతి cell లో **ఒకే (atomic) value** ఉండాలి — list/array/multiple values కాదు. ప్రతి row unique.

```
❌ 1NF కాదు (phone column లో multiple values)
+----------+---------+---------------------+
| stud_id | name    | phones              |
+----------+---------+---------------------+
|   1      | Ravi    | 99999, 88888        |  ← ఒక cell లో 2 values
+----------+---------+---------------------+

✅ 1NF (atomic — ప్రతి phone వేరే row)
+----------+---------+---------+
| stud_id | name    | phone   |
+----------+---------+---------+
|   1      | Ravi    | 99999   |
|   1      | Ravi    | 88888   |
+----------+---------+---------+
```

> **Mongo తో pole:** Mongo లో array field (`phones: [99999, 88888]`) పూర్తిగా valid, common. అంటే Mongo documents **1NF ని ఉద్దేశపూర్వకంగా ఉల్లంఘిస్తాయి** — ఇదే document DB యొక్క flexibility (కానీ joins/consistency trade-off). Relational world లో atomic తప్పనిసరి.

2NF — partial dependency తీసేయడం

నియమం: 1NF + **ఏ non-prime attribute కూడా candidate key యొక్క భాగం మీద మాత్రమే ఆధారపడకూడదు** (composite key ఉన్నప్పుడే relevant).

Example — `enrollments(stud_id, course_id, grade, course_name)` , PK = (stud_id, course_id):

```
FDs:
  (stud_id, course_id) → grade          ✅ full key మీద ఆధారపడుతుంది (OK)
  course_id → course_name               ❌ course_name కి stud_id అవసరం లేదు — partial
dependency!
```

`course_name` కేవలం `course_id` (key లో సగం) మీద ఆధారపడుతోంది → **partial dependency** → 2NF violate. Fix: విడగొట్టు.

```sql
-- 2NF: course info ని విడిగా తీయి
CREATE TABLE courses (
    course_id   INT PRIMARY KEY,
    course_name VARCHAR(100)        -- ఇప్పుడు ఒకేసారి store, redundancy లేదు
);
CREATE TABLE enrollments (
    stud_id   INT,
    course_id INT REFERENCES courses(course_id),
    grade     CHAR(2),
    PRIMARY KEY (stud_id, course_id)
);
```

3NF — transitive dependency తీసేయడం

నియమం: 2NF + **ఏ non-prime attribute మరో non-prime attribute మీద ఆధారపడకూడదు** (transitive dependency లేకుండా). అంటే non-key columns నేరుగా key మీద మాత్రమే ఆధారపడాలి.

Example — `employees(emp_id, name, dept_id, dept_name)` :

```
FDs:
  emp_id → dept_id        ✅ (key → non-prime)
  dept_id → dept_name     ❌ non-prime → non-prime → transitive!
  కాబట్టి: emp_id → dept_id → dept_name (transitive)
```

`dept_name` నేరుగా `emp_id` మీద కాక `dept_id` ద్వారా ఆధారపడుతోంది. Problem: ఒకే dept లో 100 employees ఉంటే, dept_name 100 సార్లు repeat + update anomaly. Fix:

```sql
-- 3NF: department ని విడిగా
CREATE TABLE departments (
```

```
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(100)
);
CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(100),
    dept_id INT REFERENCES departments(dept_id)   -- dept_name ఇక్కడ లేదు
);
```

3NF ని గుర్తుంచుకునే mantra: "The key, the whole key, and nothing but the key" — ప్రతి non-key column **key మీద** (1NF+valid), **whole key మీద** (2NF, partial లేదు), **key మీద మాత్రమే** (3NF, transitive లేదు) ఆధారపడాలి.

BCNF (Boyce-Codd Normal Form) — 3NF యొక్క కఠిన రూపం

నియమం: ప్రతి functional dependency `X → Y` లో, **X ఒక super key అయి ఉండాలి.** అంటే determinant (బాణం ఎడమ వైపు) ఎప్పుడూ key అవ్వాలి.

3NF vs BCNF: 3NF ఒక edge case allow చేస్తుంది — non-prime attribute, prime attribute ని determine చేసినా OK అంటుంది. BCNF దీన్ని allow చేయదు. చాలా tables లో 3NF = BCNF; తేడా వచ్చేది **multiple overlapping candidate keys** ఉన్నప్పుడే.

Example — `(student, subject, teacher)` : ఒక teacher ఒకే subject చెప్తాడు, కానీ ఒక subject ని చాలా teachers చెప్పొచ్చు; ఒక student ఒక subject కి ఒక teacher.

```
FDs:
  (student, subject) → teacher    (candidate key)
  teacher → subject               ← teacher subject ని determine చేస్తుంది, కానీ teacher super key
కాదు!
```

`teacher → subject` లో `teacher` super key కాదు → **BCNF violate** (కానీ subject prime కాబట్టి 3NF satisfy). Fix: విడగొట్టు — `teacher_subject(teacher, subject)` + `student_teacher(student, teacher)` .

Normal Forms — summary table

Normal Form	ఏం తీసేస్తుంది	ఒక్క వాక్యంలో
1NF	multi-valued/non-atomic cells	ప్రతి cell atomic
2NF	partial dependencies	non-prime, full key మీద ఆధారపడాలి
3NF	transitive dependencies	non-prime, non-prime మీద ఆధారపడకూడదు
BCNF	remaining anomalies	ప్రతి determinant super key అవ్వాలి

(తర్వాత 4NF — multi-valued dependencies, 5NF — join dependencies ఉన్నాయి కానీ interview లో అరుదు. 3NF/BCNF వరకు తెలిస్తే చాలు. **Practical target = 3NF.**)

Denormalization — ఎప్పుడు rules ని కావాలని ఉల్లంఘించాలి

Normalization redundancy తీస్తుంది కానీ **joins పెంచుతుంది** — ప్రతి query లో చాలా tables join చేయాలి → read నెమ్మది అవ్వొచ్చు. **Denormalization** = performance కోసం కావాలని కొంత redundancy తిరిగి పెట్టడం.

అంశం	Normalization	Denormalization
Redundancy	తక్కువ	ఎక్కువ (కావాలని)
Data integrity	ఎక్కువ (one place)	తక్కువ (sync అవసరం)
Write speed	వేగం (ఒక చోట update)	నెమ్మది (multiple copies)
Read speed	నెమ్మది (joins)	వేగం (joins తక్కువ)
ఎప్పుడు	OLTP (transactional, write-heavy)	OLAP (analytics, read-heavy), reporting

ఉదా: ఒక `orders` table లో `customer_name` ని duplicate గా పెట్టడం (customers తో join తప్పించడానికి). Read fast, కానీ customer name మారితే orders లో కూడా update చేయాలి.

> **Mongo తో pole:** Mongo లో denormalization = **norm** (default). Order document లోపల customer name, address embed చేయడం common — joins తప్పించడానికి. అంటే Mongo "read-optimized denormalized" thinking; RDBMS "normalized, join when needed" thinking. Topic 14 లో ఈ trade-off deep.

Key Points

- **Normalization** = redundancy + anomalies (insertion/update/deletion) తగ్గించడానికి పెద్ద table ని విభజించడం. మూలం: redundancy.
- **FD** `X → Y` = X తెలిస్తే Y fix. Prime (key లో) vs non-prime attributes.
- **1NF** (atomic cells) → **2NF** (no partial dependency) → **3NF** (no transitive dependency) → **BCNF** (ప్రతి determinant super key).
- Mantra: **"the key, the whole key, and nothing but the key" (3NF).**
- **Denormalization** = performance (read speed) కోసం కావాలని redundancy — OLAP/reporting/read-heavy కి. Practical target = 3NF, అవసరమైతే denormalize.

Interview దృష్టి

Q: Normalization ఎందుకు? Anomalies అంటే ఏమిటి? A: Data redundancy ని తగ్గించి 3 anomalies నివారించడానికి — **Insertion** (అనవసర data లేకుండా insert చేయలేకపోవడం), **Update** (ఒక fact చాలా చోట్ల update చేయాల్సి రావడం, miss అయితే inconsistency), **Deletion** (ఒక row తీస్తే వేరే అవసర data పోవడం). ఈ మూడూ redundancy వల్లే వస్తాయి; normalization "one fact, one place" enforce చేసి వీటిని తీస్తుంది.

Q: 2NF, 3NF, BCNF తేడా ఒక్క వాక్యంలో? A: **2NF** — partial dependency లేదు (non-prime attribute full composite key మీద ఆధారపడాలి, part మీద కాదు). **3NF** — transitive dependency లేదు (non-prime, మరో non-prime మీద ఆధారపడకూడదు). **BCNF** — ప్రతి determinant (X→Y లో X) super key అవ్వాలి; 3NF యొక్క కఠిన రూపం, overlapping candidate keys ఉన్నప్పుడు తేడా వస్తుంది.

Q: ఎప్పుడూ full normalize చేయాలా? A: కాదు. Normalization data integrity కి మంచిది కానీ joins పెంచి reads నెమ్మది చేస్తుంది. Read-heavy systems (analytics, dashboards, reporting) లో **denormalization** (కొంత redundancy కావాలని) performance కి better — joins తగ్గిస్తుంది. Practical rule: **3NF వరకు normalize చేయి, measured performance అవసరం ఉంటేనే denormalize చేయి** (premature denormalization అవద్దు). Trade-off: read speed vs write consistency.

Q: Denormalized data ని consistent గా ఎలా ఉంచుతావు? A: Application logic (write time లో అన్ని copies update), database triggers, లేదా periodic batch jobs/materialized views. Mongo లో ఇది manual (embed చేసిన data ని update చేయాలి). ఇది denormalization యొక్క ధర — read speed కి consistency effort.

Part 4 — Transactions (లావాదేవీలు)

> ఇది database theory యొక్క గుండె — SSE interview లో అత్యధికంగా, అత్యంత లోతుగా అడిగే భాగం. Bank లో డబ్బు transfer అవుతున్నప్పుడు మధ్యలో power పోతే? ఇద్దరు ఒకేసారి last ticket book చేస్తే? ఈ real problems ని **transactions, ACID, concurrency control, isolation levels** ఎలా solve చేస్తాయో ఇక్కడ నేర్చుకుంటాం. ఇది పక్కాగా ఉంటే నీవు senior గా నిలబడతావు.

---

9. Transactions & ACID

వివరణ

Transaction = ఒక logical unit of work — ఒకటి లేదా అంతకంటే ఎక్కువ SQL statements, అవి **అన్నీ కలిసి జరగాలి లేదా ఏదీ జరగకూడదు (all-or-nothing).** మధ్యలో సగం జరిగి ఆగకూడదు.

Classic ఉదాహరణ — bank transfer: Ravi నుండి Sita కి ₹1000 పంపడం అంటే రెండు steps:

1. Ravi account నుండి ₹1000 తీయడం (debit).
2. Sita account కి ₹1000 కలపడం (credit).

Step 1 అయ్యి, step 2 కి ముందు **power పోతే?** ₹1000 గాలిలో మాయం! Ravi నుండి పోయింది, Sita కి రాలేదు. **ఇది జరగకూడదు** — రెండూ జరగాలి, లేదా ఏదీ జరగకూడదు. ఇదే transaction ఇచ్చే guarantee.

```
BEGIN;                                                   -- transaction మొదలు
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;   -- Ravi debit
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;   -- Sita credit
COMMIT;                         -- రెండూ success → permanent. మధ్యలో fail → ROLLBACK (ఏదీ జరగదు)
```

Real-life Scenario

> **Transaction = ATM లో డబ్బు తీయడం.**
>
> నీవు ATM లో ₹5000 తీస్తున్నావు. లోపల జరిగేది:
>
> 1. నీ balance నుండి ₹5000 తీసేయడం.
> 2. Machine నుండి cash బయటకు రావడం.
>
> ఇప్పుడు — balance తగ్గింది కానీ **machine లో cash ఇరుక్కుపోయింది** (jam)? నీవు ₹5000 పోగొట్టుకున్నావు, cash రాలేదు! ఇది ఎప్పుడూ జరగకూడదు.
>
> అందుకే bank దీన్ని ఒక **transaction** గా చూస్తుంది — cash successfully బయటకు వస్తేనే balance తగ్గుతుంది (COMMIT). Cash jam అయితే — balance తిరిగి మామూలుగా (ROLLBACK), నీవు ఏమీ పోగొట్టుకోవు. **All-or-nothing.**

ACID — transactions యొక్క 4 guarantees (అత్యంత ముఖ్యం)

ప్రతి valid transaction ఈ 4 properties ని satisfy చేయాలి. **ACID** = Atomicity, Consistency, Isolation, Durability. Interview లో ప్రతి ఒక్కటి example తో explain చేయగలగాలి.

అక్షరం	Property	అర్థం	ఉదాహరణ
A	**Atomicity**	All-or-nothing. అన్ని steps జరగాలి లేదా ఏదీ జరగకూడదు	Transfer లో debit జరిగి credit fail → మొత్తం rollback
C	**Consistency**	Transaction DB ని ఒక valid state నుండి మరో valid state కి తీసుకెళ్తుంది (constraints/rules break అవ్వవు)	Transfer ముందు, తర్వాత total money same. Constraints (balance ≥ 0) hold
I	**Isolation**	Concurrent transactions ఒకదానికొకటి interfere అవ్వకూడదు (ఒంటరిగా run అయినట్లు)	ఇద్దరు ఒకేసారి transfer చేస్తే, ఒకరి half-done state మరొకరికి కనిపించకూడదు
D	**Durability**	COMMIT అయ్యాక data శాశ్వతం — power/crash అయినా పోదు	Transfer COMMIT అయ్యాక server crash అయినా, restart తర్వాత డబ్బు అక్కడే ఉంటుంది

ఒక్కొక్కటీ లోతుగా:

Atomicity (అణుత్వం) — "అన్నీ లేదా ఏదీ లేదు" DBMS దీన్ని **transaction log / undo log** తో సాధిస్తుంది. ఏదైనా step fail అయితే, అప్పటివరకు జరిగినవి undo (rollback) చేస్తుంది. COMMIT లేదా ROLLBACK — మధ్య state ఉండదు.

```sql
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
-- ఇక్కడ error (ఉదా: Sita account లేదు, or constraint fail)
ROLLBACK;     -- Ravi debit కూడా undo → ఏమీ జరగనట్లు
```

Consistency (స్థిరత్వం) — "valid → valid" Transaction అన్ని rules (constraints, foreign keys, triggers) ని గౌరవించాలి. ఉదా: ఒక `CHECK (balance >= 0)` ఉంటే, balance negative అయ్యే transaction fail అవుతుంది. Application invariants (total money constant) కూడా consistency లో భాగం. **గమనిక:** ఇది application + DB కలిసి ensure చేసేది.

Isolation (వేరుపాటు) — "ఒంటరిగా run అయినట్లు" అనేక transactions ఒకేసారి run అయినా, ఫలితం అవి వరుసగా (serial) run అయినట్లు ఉండాలి. కానీ full isolation నెమ్మది → **isolation levels** అనే tunable trade-off ఉంది (Topic 10 — dirty reads etc.). ఇది ACID లో అత్యంత subtle, interview-heavy భాగం.

Durability (మన్నిక) — "COMMIT = శాశ్వతం" COMMIT return అయ్యాక, data disk మీద safe — power failure, crash అయినా పోదు. DBMS దీన్ని **WAL (Write-Ahead Logging)** తో సాధిస్తుంది — data pages disk కి write అవ్వకముందే, change ని log కి write చేస్తుంది (Topic 13). Crash తర్వాత log నుండి recover.

TCL commands — transaction ని control చేయడం

```sql
BEGIN;                      -- (లేదా START TRANSACTION) transaction మొదలు
-- ... statements ...
SAVEPOINT sp1;              -- ఒక checkpoint (partial rollback కి)
-- ... more statements ...
```

```
ROLLBACK TO sp1;            -- sp1 వరకు undo (అంతకుముందువి ఉంటాయి)
COMMIT;                     -- మొత్తం permanent
-- లేదా
ROLLBACK;                   -- మొత్తం undo
```

- **COMMIT** — transaction ని permanent చేస్తుంది. తర్వాత undo కుదరదు.
- **ROLLBACK** — transaction మొదలు నుండి అన్ని changes undo.
- **SAVEPOINT** — transaction లోపల ఒక marker; ఆ point వరకు మాత్రమే rollback చేయవచ్చు (partial).

> **Auto-commit gotcha:** చాలా DB clients default గా **auto-commit** mode లో ఉంటాయి — ప్రతి statement వెంటనే commit అవుతుంది (implicit transaction). Multi-statement transaction కావాలంటే explicit `BEGIN` వాడాలి. ఇది తెలియకపోతే "నా ROLLBACK పనిచేయలేదు" అనే confusion వస్తుంది.

Transaction states — lifecycle

```
┌────────┐   begin    ┌───────────┐  last stmt  ┌──────────────────┐
│ Active │──────────>│ Partially │───────────>│ Committed        │
└────────┘           │ Committed │   COMMIT    │ (permanent)      │
    │                └───────────┘             └──────────────────┘
    │ error/fail           │ error
    ▼                      ▼
┌────────┐  rollback  ┌────────────┐
│ Failed │──────────>│ Aborted    │ (rolled back, DB unchanged)
└────────┘           └────────────┘
```

Key Points

- **Transaction = logical unit of work, all-or-nothing.** `BEGIN ... COMMIT / ROLLBACK`.
- **ACID: A**tomicity (all-or-nothing, undo log), **C**onsistency (valid→valid, constraints hold), **I**solation (concurrent = serial-లా, tunable), **D**urability (COMMIT = శాశ్వతం, WAL).
- **Atomicity** transaction log తో, **Durability** WAL తో సాధిస్తారు.
- **SAVEPOINT** = partial rollback marker. **Auto-commit** default ⇒ multi-statement కి explicit BEGIN.
- Isolation = ACID లో అత్యంత subtle → isolation levels తో tune (Topic 10).

Interview దృష్టి

Q: ACID properties ఏమిటి, ప్రతిదీ example తో? A: **Atomicity** — transaction అంతా జరగాలి లేదా ఏదీ జరగకూడదు (bank transfer: debit జరిగి credit fail → రెండూ rollback). **Consistency** — DB ఒక valid state నుండి మరో valid state కి (constraints, total money hold). **Isolation** — concurrent transactions interfere అవ్వకూడదు (serial-లా కనిపించాలి). **Durability** — COMMIT తర్వాత crash అయినా data ఉంటుంది (WAL వల్ల). ఈ 4 కలిసి reliable transactions ఇస్తాయి.

Q: Atomicity ఎలా achieve అవుతుంది? A: DBMS **undo log / transaction log** maintain చేస్తుంది — ప్రతి change కి before-image (పాత value) log అవుతుంది. Transaction fail/rollback అయితే, log నుండి పాత values తిరిగి రాసి undo చేస్తుంది. COMMIT అయ్యేవరకు changes "tentative"; fail అయితే అన్నీ reverse.

Q: Durability ఎలా — data disk కి write అవ్వకముందే crash అయితే? A: **WAL (Write-Ahead Logging).** COMMIT ముందు, change ని ముందు **log file కి** (append-only, fast, sequential write) రాసి disk కి flush చేస్తారు — actual data page తర్వాత రాయవచ్చు. Crash అయితే, restart time లో WAL ని replay చేసి committed changes తిరిగి apply చేస్తారు (redo). అందుకే COMMIT return అయితే data guaranteed safe, data page ఇంకా disk కి వెళ్ళకపోయినా.

Q: COMMIT vs ROLLBACK vs SAVEPOINT? A: **COMMIT** transaction changes ని permanent చేస్తుంది. **ROLLBACK** మొత్తం transaction ని undo చేస్తుంది. **SAVEPOINT** transaction లోపల intermediate marker — `ROLLBACK TO savepoint` తో ఆ point వరకే undo చేసి, మిగతా transaction కొనసాగించవచ్చు (partial rollback, complex transactions లో error recovery కి).

10. Concurrency Control (isolation, locks, 2PL)

వివరణ

ఒకేసారి (concurrently) చాలా transactions run అయినప్పుడు, అవి ఒకే data ని touch చేస్తే **సమస్యలు** వస్తాయి. **Concurrency control** = ఈ transactions ని safe గా, correct results ఇచ్చేలా manage చేసే mechanism. ఇది ACID లోని **Isolation** ని అమలు చేసే విభాగం.

ఎందుకు ముఖ్యం? Isolation ని 100% పాటిస్తే (transactions వరుసగా run) — correct కానీ **నెమ్మది** (no parallelism). Isolation ని loosen చేస్తే — **వేగం** కానీ కొన్ని anomalies వస్తాయి. ఈ trade-off ని control చేసేదే **isolation levels.**

Real-life Scenario

> **Concurrency problem = ఒకే cinema seat ఇద్దరికి book అవడం.**
>
> BookMyShow లో last seat (A5) ఉంది. Ravi, Sita ఇద్దరూ **ఒకేసారి** ఆ seat select చేశారు:
>
> 1. Ravi చూస్తాడు — "A5 available" ✅
> 2. Sita చూస్తుంది — "A5 available" ✅ (Ravi ఇంకా book చేయలేదు)
> 3. Ravi books A5.
> 4. Sita కూడా books A5!
>
> ఇప్పుడు **ఒకే seat ఇద్దరికి** — theatre లో గొడవ. ఇది concurrency లేకపోతే వచ్చే problem. Solution: Ravi seat select చేయగానే **lock** వేయడం — Sita ఆ seat ని Ravi book/release చేసేవరకు touch చేయలేదు. ఇదే **locking.**

Concurrency సమస్యలు (Read Phenomena) — interview core

Isolation సరిగ్గా లేకపోతే 3 (+1) రకాల anomalies వస్తాయి. వీటిని పేరుతో, example తో గుర్తుంచుకో:

1. Dirty Read — commit అవ్వని data ని చదవడం

```
T1: UPDATE balance = 500 (ఇంకా commit కాలేదు)
T2:                         READ balance → 500 (dirty! T1 rollback అయితే ఈ 500 తప్పు)
T1: ROLLBACK                ← ఇప్పుడు T2 చదివిన 500 అసలు ఉనికిలో లేని value
```

T2 ఒక "uncommitted, తర్వాత undo అయ్యే" value చదివింది. ప్రమాదకరం.

2. Non-Repeatable Read — అదే row ని రెండుసార్లు చదివితే వేరే values

```
T1: READ balance → 500
T2: UPDATE balance = 800, COMMIT
T1: READ balance → 800       ← అదే transaction లో అదే row, వేరే value! (repeatable కాదు)
```

T1 ఒకే query రెండుసార్లు run చేస్తే వేరే result — inconsistent.

3. Phantom Read — అదే condition query కి కొత్త rows కనిపించడం

```
T1: SELECT COUNT(*) FROM orders WHERE amount > 100 → 5 rows
T2: INSERT INTO orders (amount=200), COMMIT
T1: SELECT COUNT(*) FROM orders WHERE amount > 100 → 6 rows   ← కొత్త "phantom" row!
```

Non-repeatable read individual row గురించి; phantom **కొత్త rows (set మారడం)** గురించి.

4. Lost Update — ఇద్దరు ఒకే value మార్చి, ఒకరిది పోవడం

```
T1: READ count=10 ... write count=11
T2: READ count=10 ... write count=11    ← T1 update పోయింది! (11 అవ్వాలి, 12 అవ్వాలి నిజానికి)
```

Isolation Levels — anomalies vs performance trade-off (అత్యంత ముఖ్యం)

SQL standard 4 isolation levels ఇస్తుంది. పైకి వెళ్ళే కొద్దీ **safer కానీ slower.** ఈ table interview లో గీయగలగాలి:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	వేగం
READ UNCOMMITTED	✅ జరగొచ్చు	✅ జరగొచ్చు	✅ జరగొచ్చు	అత్యధిక వేగం
READ COMMITTED	❌ ఆగుతుంది	✅ జరగొచ్చు	✅ జరగొచ్చు	వేగం
REPEATABLE READ	❌	❌ ఆగుతుంది	✅ జరగొచ్చు*	మధ్యస్థం
SERIALIZABLE	❌	❌	❌ ఆగుతుంది	నెమ్మది (అత్యంత safe)

(MySQL InnoDB లో REPEATABLE READ phantom ని కూడా ఆపుతుంది — gap locks / MVCC వల్ల. PostgreSQL లో REPEATABLE READ = snapshot isolation, phantoms ఆగుతాయి. Standard vs implementation తేడా — interview లో mention చేస్తే bonus.)

Defaults: PostgreSQL, Oracle, SQL Server → **READ COMMITTED.** MySQL (InnoDB) → **REPEATABLE READ.** ఇవి తెలిస్తే practical depth కనిపిస్తుంది.

```
-- isolation level set చేయడం
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN;
SELECT balance FROM accounts WHERE id = 1;
```

```
-- ...
COMMIT;
```

ఎప్పుడు ఏది?

- **READ COMMITTED** — చాలా apps కి default, మంచి balance (dirty reads ఆగుతాయి, decent speed).
- **SERIALIZABLE** — financial, inventory (last-seat problem) లాంటి strict correctness అవసరమైన చోట. Cost: contention, retries.
- **READ UNCOMMITTED** — దాదాపు ఎప్పుడూ వాడరు (analytics లో approximate counts తప్ప).

Locks — concurrency ని అమలు చేసే tool

Lock = ఒక transaction ఒక resource (row/table) మీద వేసే "నా control లో ఉంది" mark. రెండు ప్రధాన రకాలు:

Lock రకం	పేరు	ఎవరు access చేయవచ్చు
Shared (S)	Read lock	చాలామంది ఒకేసారి read చేయవచ్చు, ఎవరూ write చేయలేరు
Exclusive (X)	Write lock	ఒక్కరే — read/write ఎవరూ చేయలేరు

Lock compatibility: S+S ✅ (ఇద్దరూ read ok), S+X ❌, X+X ❌ (write exclusive).

```
-- explicit locking (Postgres/MySQL): row ని lock చేసి చదవడం
BEGIN;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;   -- exclusive lock → ఇతరులు wait
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
COMMIT;                                             -- lock release
-- FOR SHARE = shared lock (ఇతరులు read ok, write కాదు)
```

Lock granularity: row-level (fine, ఎక్కువ concurrency, ఎక్కువ overhead) → page → table-level (coarse, తక్కువ concurrency, తక్కువ overhead). చాలా DBs row-level locks వాడతాయి.

2PL (Two-Phase Locking) — serializability ని guarantee చేసే protocol

2PL = ఒక transaction locks ని రెండు phases లో handle చేస్తుంది — దీంతో serializability guarantee అవుతుంది:

```
      locks
        │       ┌────── (growing) ──────┐ ┌─── (shrinking) ───┐
count   │      /                         \/                    \
        │     /  locks acquire చేస్తుంది      \  locks release చేస్తుంది
        │    /   (ఏదీ release చేయదు)          \  (ఏదీ acquire చేయదు)
        └────┴───────────────────────────┴──────────────────────→ time
             Growing Phase        lock point      Shrinking Phase
```

- **Growing phase:** transaction locks acquire చేస్తుంది, ఏదీ release చేయదు.
- **Shrinking phase:** locks release చేయడం మొదలుపెట్టాక, కొత్త lock acquire చేయకూడదు.

Strict 2PL (practical variant): అన్ని **exclusive locks ని COMMIT/ROLLBACK వరకు** hold చేస్తుంది → dirty reads + cascading rollbacks ఆగుతాయి. చాలా DBs దీన్ని (లేదా MVCC) వాడతాయి.

MVCC — locks లేకుండా reads (modern approach)

MVCC (Multi-Version Concurrency Control) — PostgreSQL, MySQL InnoDB, Oracle వాడే modern technique. Idea: ప్రతి row యొక్క **multiple versions** ఉంచడం. ఒక transaction write చేస్తే **కొత్త version** create అవుతుంది; readers **పాత version (snapshot)** చదువుతారు.

Benefit: Readers writers ని block చేయరు, writers readers ని block చేయరు — huge concurrency win. Write-write conflict కి మాత్రమే locks. అందుకే modern DBs లో reads చాలా fast. (పాత versions ని `VACUUM` /garbage collection తీసేస్తుంది.)

> **Mongo తో pole:** MongoDB కూడా document-level locking + WiredTiger storage engine లో MVCC-లాంటి snapshot isolation వాడుతుంది. Multi-document transactions (Mongo 4.0+) కి కూడా isolation ఉంది. కానీ Mongo design embedding-first కాబట్టి, చాలా operations single-document (atomic by default) — cross-document transactions అంత common కాదు RDBMS లో లాగా.

Key Points

- **Read phenomena: Dirty read** (uncommitted చదవడం), **non-repeatable read** (అదే row వేరే value), **phantom read** (కొత్త rows), **lost update.**
- **Isolation levels (safe↑, fast↓):** READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE. Postgres default = READ COMMITTED, MySQL = REPEATABLE READ.
- **Locks:** Shared (S, read, multiple ok) vs Exclusive (X, write, ఒక్కరే). `SELECT ... FOR UPDATE`.
- **2PL** (growing/shrinking phases) serializability guarantee చేస్తుంది; **Strict 2PL** locks ని COMMIT వరకు hold.
- **MVCC** = row versions → readers/writers ఒకరినొకరు block చేయరు (modern DBs, huge concurrency win).

Interview దృష్టి

Q: Dirty read, non-repeatable read, phantom read తేడా? A: **Dirty read** — మరో transaction యొక్క **uncommitted** change ని చదవడం (అది rollback అయితే మనం చదివింది invalid). **Non-repeatable read** — అదే transaction లో అదే **row** ని రెండుసార్లు చదివితే వేరే value (మధ్యలో వేరే transaction commit చేసింది). **Phantom read** — అదే **condition** query కి రెండోసారి **కొత్త rows** కనిపించడం (మధ్యలో insert/delete). మొదటిది uncommitted; రెండోది existing row మార్పు; మూడోది row set మార్పు.

Q: Isolation levels ఏవి, ఏది ఏ anomaly ఆపుతుంది? A: READ UNCOMMITTED (ఏదీ ఆపదు) → READ COMMITTED (dirty read ఆపుతుంది) → REPEATABLE READ (+ non-repeatable ఆపుతుంది) → SERIALIZABLE (+ phantom ఆపుతుంది, పూర్తి isolation). పైకి వెళ్ళే కొద్దీ safer కానీ slower (ఎక్కువ locking/contention). Practical default READ COMMITTED; strict correctness (banking, inventory) కి SERIALIZABLE.

Q: 2PL అంటే ఏమిటి, ఎందుకు? A: Two-Phase Locking — transaction ముందు అన్ని locks acquire చేస్తుంది (growing phase), తర్వాత release చేయడం మొదలుపెడుతుంది (shrinking phase), release మొదలయ్యాక కొత్త lock తీసుకోదు. ఇది **serializability** (concurrent transactions serial-లా ఉంటాయి) guarantee చేస్తుంది. **Strict 2PL** exclusive locks ని COMMIT వరకు hold చేసి dirty reads/cascading aborts ఆపుతుంది.

Q: MVCC readers ని ఎలా fast చేస్తుంది? A: ప్రతి row యొక్క multiple versions ఉంచడం ద్వారా. Writer కొత్త version create చేస్తుంది, పాత version అలాగే ఉంటుంది. Readers commit అయిన సమయం snapshot (పాత version) చదువుతారు — writer యొక్క in-progress change కోసం wait అవ్వరు. అంటే **reads locks తీసుకోవు, writers reads ని block చేయరు** → చాలా ఎక్కువ concurrency. Postgres, MySQL InnoDB, Oracle దీన్ని వాడతాయి.

11. Deadlocks in DB

వివరణ

Deadlock = రెండు (లేదా అంతకంటే ఎక్కువ) transactions, ఒకదానికొకటి hold చేసిన locks కోసం **శాశ్వతంగా wait** చేసే పరిస్థితి — ఏదీ ముందుకు సాగదు. Circular waiting.

Classic ఉదాహరణ:

```
T1: locks row A ... ఇప్పుడు row B కావాలి (కానీ B ని T2 hold చేసింది) → wait
T2: locks row B ... ఇప్పుడు row A కావాలి (కానీ A ని T1 hold చేసింది) → wait
```

T1, T2 ఇద్దరూ ఒకరికొకరు కోసం wait — ఎప్పటికీ తెగదు. ఇదే deadlock.

Real-life Scenario

> **Deadlock = ఇరుకు వంతెన మీద ఎదురెదురు కార్లు.**
>
> ఒక ఇరుకు వంతెన — ఒక కారు మాత్రమే పట్టేంత. తూర్పు నుండి Ravi కారు సగం వచ్చింది, పడమర నుండి Sita కారు సగం వచ్చింది. ఇద్దరూ ఎదురెదురుగా, మధ్యలో ఇరుక్కున్నారు.
>
> - Ravi ముందుకు వెళ్ళాలంటే Sita వెనక్కి వెళ్ళాలి.
> - Sita ముందుకు వెళ్ళాలంటే Ravi వెనక్కి వెళ్ళాలి.
> - ఇద్దరూ "నువ్వు ముందు వెనక్కి వెళ్ళు" అని పట్టుబట్టారు → **ఎవరూ కదలరు. Deadlock.**
>
> Solution: ఎవరో ఒకరు (traffic police చెప్తే) వెనక్కి వెళ్ళాలి (ఒక transaction ని **abort/rollback** చేయడం = "victim"). లేదా ముందే rule — "ఎప్పుడూ కారు తక్కువ ఉన్నవాడు వెనక్కి" (deadlock prevention).

Deadlock యొక్క 4 conditions (Coffman conditions)

Deadlock జరగాలంటే **ఈ 4 conditions అన్నీ** ఏకకాలంలో ఉండాలి. ఏ ఒక్కటి తీసేసినా deadlock ఆగుతుంది — ఇదే prevention ఆధారం:

Condition	అర్థం
Mutual Exclusion	Resource ని ఒక్కరే exclusive గా hold చేయగలరు (shared కాదు)
Hold and Wait	ఒక lock hold చేస్తూనే మరో lock కోసం wait
No Preemption	Lock ని బలవంతంగా తీసుకోలేం; hold చేసినవాడే release చేయాలి
Circular Wait	T1→T2→T3→...→T1 అనే wait circle

Deadlock ని ఎలా handle చేస్తారు — 3 approaches

1. Deadlock Detection (గుర్తించి, పరిష్కరించడం) — చాలా DBs వాడేది

DBMS ఒక **wait-for graph** maintain చేస్తుంది — ఏ transaction ఏ దాని కోసం wait చేస్తోందో. ఈ graph లో **cycle** ఉంటే → deadlock. అప్పుడు ఒక transaction ని **victim** గా ఎంచుకుని **rollback** (abort) చేస్తుంది → cycle తెగుతుంది, మిగతావి కొనసాగుతాయి.

```
wait-for graph:    T1 ──waits──> T2
                   ▲             │
                   └────waits────┘      ← cycle! deadlock. → ఒకదాన్ని kill
```

Victim selection: సాధారణంగా **least work done / cheapest to rollback** transaction ని ఎంచుకుంటారు.

```
-- Postgres/MySQL deadlock detect చేస్తే, ఒక transaction కి error వస్తుంది:
-- ERROR: deadlock detected  (Postgres)
-- ERROR 1213: Deadlock found when trying to get lock (MySQL)
-- App ఈ error ని catch చేసి transaction ని retry చేయాలి.
```

2. Deadlock Prevention (అసలు జరగకుండా ఆపడం)

4 conditions లో ఏదో ఒకటి భగ్నం చేయడం:

- **Lock ordering (అత్యంత practical):** అన్ని transactions resources ని **ఒకే order** లో lock చేస్తే circular wait అసాధ్యం. ఉదా: ఎప్పుడూ చిన్న id ముందు lock చేయి — transfer లో min(id1,id2) ముందు.
- **Timeout-based (Wait-Die / Wound-Wait):** transaction timestamp ఆధారంగా — పాత transaction wait చేస్తుంది, కొత్తది die అవుతుంది (లేదా reverse). ఇది circular wait ని ఆపుతుంది.

3. Deadlock Avoidance

Resource allocation ముందే check చేసి, deadlock వచ్చే allocation ని అనుమతించకపోవడం (Banker's algorithm — OS లో ఎక్కువ, DBs లో అరుదు).

Approach	ఎప్పుడు	Cost
Detection	Deadlocks అరుదైతే (most DBs)	Detect + rollback + retry
Prevention	Deadlocks తరచుగా అయితే	Design discipline (lock ordering)
Avoidance	Resources ముందే తెలిస్తే	Runtime overhead (అరుదు)

App-level లో deadlocks ని ఎలా తగ్గించాలి (SSE practical)

1. **Consistent lock ordering** — ప్రతి చోట resources ని ఒకే order లో access చేయి (అత్యంత ముఖ్యం).
2. **Transactions చిన్నగా, వేగంగా** ఉంచు — locks తక్కువ సేపు hold → conflict తక్కువ.
3. **Retry logic** — deadlock error వస్తే, transaction ని automatic గా retry చేయి (deadlocks transient).
4. అనవసర locking తగ్గించు — `SELECT FOR UPDATE` అవసరమైన చోట.
5. Lower isolation level (అవసరమైతే) — SERIALIZABLE ఎక్కువ locking → ఎక్కువ deadlocks.

```sql
-- ✅ Lock ordering: ఎప్పుడూ చిన్న id ముందు — deadlock రాదు
BEGIN;
-- transfer between id=1 and id=2: ఎప్పుడూ 1 (చిన్నది) ముందు lock
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- రెండు concurrent transfers (1→2 మరియు 2→1) కూడా ఎప్పుడూ id=1 ముందు lock వేస్తే, circular wait
అసాధ్యం
```

Deadlock vs Livelock vs Starvation

పదం	అర్థం
Deadlock	Transactions శాశ్వతంగా ఒకరికొకరు wait, ఏదీ కదలదు (stuck)
Livelock	Transactions active గా ఉన్నాయి కానీ progress లేదు (పదేపదే retry, ఢీకొనడం)
Starvation	ఒక transaction ఎప్పుడూ resource పొందలేదు (ఇతరులు priority లో ముందుకు)

Key Points

- **Deadlock** = circular waiting for locks; ఏ transaction ముందుకు సాగదు.
- **4 Coffman conditions:** mutual exclusion, hold-and-wait, no preemption, circular wait. ఏ ఒక్కటి తీసేసినా deadlock ఆగుతుంది.
- **Detection** (wait-for graph లో cycle → victim rollback) = most DBs default. App error catch చేసి **retry** చేయాలి.
- **Prevention:** consistent **lock ordering** (అత్యంత practical), timeout (wait-die/wound-wait).
- App tips: consistent lock order, చిన్న transactions, retry logic, అనవసర locks తగ్గించడం.

Interview దృష్టి

Q: Deadlock అంటే ఏమిటి, database ఎలా handle చేస్తుంది? A: రెండు+ transactions ఒకరు hold చేసిన lock కోసం మరొకరు wait చేస్తూ circular గా ఇరుక్కోవడం — ఏదీ ముందుకు సాగదు. చాలా DBs **deadlock detection** వాడతాయి — wait-for graph maintain చేసి, cycle కనిపిస్తే ఒక **victim transaction ని rollback** చేసి cycle తెంచుతాయి. అప్పుడు application కి error వస్తుంది, దాన్ని catch చేసి transaction ని **retry** చేయాలి.

Q: Deadlock ని ఎలా prevent చేస్తావు? A: అత్యంత practical మార్గం — **consistent lock ordering.** అన్ని transactions resources ని ఒకే order (ఉదా: ascending id) లో lock చేస్తే circular wait అసాధ్యం → deadlock రాదు. అదనంగా: transactions చిన్నగా ఉంచడం (locks తక్కువ సేపు), అనవసర locking తగ్గించడం, retry logic. Theory: Coffman conditions లో ఏదో ఒకటి భగ్నం చేయడం (ఇక్కడ circular wait).

Q: Deadlock detection vs prevention ఎప్పుడు ఏది? A: **Detection** — deadlocks అరుదు అనుకుంటే (most OLTP systems), let them happen, detect చేసి rollback+retry. Overhead తక్కువ common case లో. **Prevention** — deadlocks తరచుగా అయ్యే high-contention systems లో, design discipline (lock ordering) తో అసలు రాకుండా. చాలా DBs detection default, prevention ని app design లో పెడతారు.

Part 5 — Performance (పనితీరు)

> నీ query correct గా ఉంది కానీ **నెమ్మది.** ఎందుకు? ఎలా fast చేయాలి? దీనికి జవాబే **indexing** — databases లో అత్యంత high-impact, అత్యంత interview-heavy performance topic. తర్వాత query optimizer, storage internals (pages, buffer pool, WAL) చూద్దాం. ఈ Part నీ SQL ని "పని చేస్తుంది" నుండి "వేగంగా scale అవుతుంది" కి తీసుకెళ్తుంది.

---

12. Indexing

వివరణ

Index = ఒక table లో data ని వేగంగా వెతకడానికి database maintain చేసే separate data structure. Index లేకపోతే, ఒక row వెతకాలంటే database **మొత్తం table ని scan** చేయాలి (full table scan — O(n)). Index ఉంటే, నేరుగా వెళ్తుంది (O(log n)). ఇది databases performance లో అత్యంత high-impact concept — interview లో దాదాపు తప్పనిసరి.

Real-life Scenario

> **Index = పుస్తకం చివర ఉండే alphabetical index.**
>
> 500 pages పుస్తకంలో "transaction" అనే పదం ఎక్కడ ఉందో కావాలి. రెండు మార్గాలు:
>
> - **Index లేకుండా:** page 1 నుండి 500 వరకు ప్రతి page చదవాలి → 500 pages (full table scan, O(n)).
> - **Index తో:** చివర index కి వెళ్ళి "T → transaction → page 245" చూసి, నేరుగా page 245 కి → 2 steps (index lookup, O(log n)).
>
> Index అనేది exactly ఇదే — actual data కి ఒక **sorted pointer/shortcut.** వెతకడం చాలా వేగం. కానీ index కూడా space తీసుకుంటుంది, మరియు పుస్తకం మారితే index కూడా update చేయాలి (write cost). ఇదే index యొక్క core trade-off.

Index లేకుండా vs తో — demo

```sql
CREATE TABLE users (
    id    INT PRIMARY KEY,        -- PK మీద index automatic
    email VARCHAR(100),
    city  VARCHAR(50)
);
-- 10 lakh rows ఉన్నాయనుకో

-- index లేకుండా: మొత్తం 10 lakh rows scan (slow)
SELECT * FROM users WHERE email = 'ravi@mail.com';   -- full table scan

-- index create చేస్తే:
CREATE INDEX idx_users_email ON users(email);
```

```
-- ఇప్పుడు అదే query → index lookup (fast, log n)
SELECT * FROM users WHERE email = 'ravi@mail.com';
```

B-Tree / B+Tree — indexes ఎలా పనిచేస్తాయి (internals)

చాలా RDBMS indexes **B+Tree** అనే balanced tree data structure వాడతాయి. ఎందుకు plain binary tree కాదు?

- **Binary tree** — ప్రతి node కి 2 children → tree ఎత్తు ఎక్కువ → ఎక్కువ disk reads.
- **B-Tree/B+Tree** — ప్రతి node కి **చాలా children** (100s) → tree "flat, wide" → చాలా తక్కువ levels → **తక్కువ disk reads** (disk read = ఖరీదు). 10 lakh rows కి కేవలం 3-4 levels.

B+Tree specifics (B-Tree కంటే ఎందుకు better):

- **అన్ని actual data/pointers leaf nodes లోనే** (internal nodes కేవలం navigation keys). → internal nodes చిన్నవి → ఎక్కువ keys per node → tree మరింత flat.
- **Leaf nodes linked list గా connected** → **range queries** (`BETWEEN` , `>` , `ORDER BY`) చాలా fast — ఒక leaf నుండి పక్క leaf కి నడిస్తే చాలు.

```
B+Tree (simplified):
                    [ 50 | 100 ]                   ← root (navigation only)
                  /      |       \
           [20|35]    [70|85]    [120|150]         ← internal (navigation)
           /  |  \     ...
     [10..][20..][35..] ──→ [50..] ──→ [70..] ──→  ← leaves: actual data + linked (range
fast)
```

Lookup: root → internal → leaf, O(log n) ≈ 3-4 disk reads for millions of rows. **ఇదే indexes వేగానికి కారణం.**

Clustered vs Non-Clustered Index — అత్యంత ముఖ్యమైన distinction

అంశం	Clustered Index	Non-Clustered Index (Secondary)
ఏమిటి	Table యొక్క **actual rows** ఇదే order లో physically stored	Separate structure, **rows కి pointers** కలిగి ఉంటుంది
ఎన్ని per table	**ఒక్కటే** (data ఒకే physical order లో ఉండగలదు)	**చాలా** ఉండొచ్చు
Analogy	పుస్తకం content ఏ order లో ఉందో (chapters)	పుస్తకం చివర index (pointers)
Speed	కొంచెం fast (data నేరుగా అక్కడే)	ఒక extra hop (pointer → actual row)
సాధారణంగా	**Primary key** మీద (auto)	మనం create చేసే indexes

- **Clustered:** table rows ని physically sort చేసి store చేస్తుంది. అందుకే **ఒక్కటే** ఉండగలదు (data ఒకే order లో మాత్రమే ఉండగలదు). MySQL InnoDB లో PK ఎప్పుడూ clustered.
- **Non-clustered:** actual data వేరుగా ఉంటుంది; index లో sorted keys + row pointers ఉంటాయి. Lookup → index లో key కనుక్కో → pointer follow చేసి actual row తీసుకో.

> **PostgreSQL గమనిక:** Postgres లో అన్నీ technically non-clustered (heap + separate indexes); `CLUSTER` command తో one-time physical sort చేయవచ్చు కానీ maintain అవ్వదు. MySQL InnoDB, SQL Server లో clustered concept strong. Interview లో ఈ nuance mention చేస్తే depth కనిపిస్తుంది.

Composite Index & column order — subtle కానీ ముఖ్యం

Composite (multi-column) index = అనేక columns మీద ఒక index. **Column order అత్యంత ముఖ్యం** — "leftmost prefix" rule:

```
CREATE INDEX idx_city_age ON users(city, age);    -- order: city ముందు, age తర్వాత
```

ఈ index వీటికి పనిచేస్తుంది:

- `WHERE city = 'X'` ✅ (leftmost)
- `WHERE city = 'X' AND age = 25` ✅ (both, in order)
- `WHERE city = 'X' AND age > 20` ✅

కానీ వీటికి **పనిచేయదు** (సరిగ్గా):

- `WHERE age = 25` ❌ (city ని skip చేసింది — leftmost prefix లేదు)

నియమం: composite index `(A, B, C)` — queries A తో మొదలవ్వాలి. అందుకే **equality columns ముందు, range columns తర్వాత**, most-selective columns ముందు పెట్టడం best practice. (Analogy: phone directory పేరు+ఇంటిపేరుతో sort — ఇంటిపేరు మాత్రమే తెలిస్తే వెతకడం కష్టం.)

Covering Index — index-only scan

Covering index = query కి కావలసిన అన్ని columns index లోనే ఉంటే, database **actual table ని touch చేయకుండా** index నుండే answer ఇస్తుంది (index-only scan) → చాలా fast.

```
CREATE INDEX idx_cover ON orders(customer_id, amount);
-- ఈ query కి కావలసినవి (customer_id, amount) రెండూ index లో ఉన్నాయి → table చూడనవసరం లేదు
SELECT amount FROM orders WHERE customer_id = 10;   -- index-only scan (covering)
```

ఎప్పుడు index వేయాలి / ఎప్పుడు వేయకూడదు (SSE judgment)

Index వేయాలి:

- తరచుగా `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY` లో వాడే columns.
- Foreign key columns (joins fast).
- High **selectivity** columns (చాలా unique values — email, user_id).

Index వేయకూడదు / జాగ్రత్త:

- **Low selectivity** columns (gender, is_active — కొన్నే unique values). Index ఇక్కడ full scan కంటే better కాదు.
- చిన్న tables (full scan అప్పటికే fast).

- **Write-heavy** tables — ప్రతి INSERT/UPDATE/DELETE అన్ని indexes ని కూడా update చేయాలి → writes నెమ్మది. Over-indexing = write penalty.
- అరుదుగా వాడే columns.

Index యొక్క trade-off (interview core):

ప్రయోజనం	ధర
Reads వేగం (SELECT/WHERE/JOIN)	Writes నెమ్మది (INSERT/UPDATE index update)
Sorting/range fast	Extra storage (index data structure)
—	Maintenance overhead (rebuild, fragmentation)

ఒక్క వాక్యంలో: Index = **read speed ని write speed + storage తో కొనడం.** Read-heavy కి worth; write-heavy లో balance.

EXPLAIN — index వాడబడిందా చూడటం

`EXPLAIN` (Postgres: `EXPLAIN ANALYZE`) query యొక్క **execution plan** చూపిస్తుంది — index వాడిందా, ఎన్ని rows scan అయ్యాయి, cost ఎంత. Performance debug చేయడానికి essential tool.

```
EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'ravi@mail.com';
```

Output లో చూడాల్సినవి:

- `Seq Scan` (Postgres) / `type: ALL` (MySQL) → **full table scan** (index వాడలేదు — bad, index కావాలి).
- `Index Scan` / `type: ref/range` → index వాడింది (good).
- **rows** — ఎన్ని rows examine అయ్యాయి (తక్కువ = better).
- **cost / actual time** — estimated/actual expense.

> **Index వాడబడని common కారణాలు:** column మీద function (`WHERE UPPER(email) = ...` → index బద్ధలవుతుంది; functional index వాడాలి), leading wildcard (`LIKE '%x'`), type mismatch, low selectivity (optimizer full scan choose చేస్తుంది), stale statistics.

Index రకాలు (brief)

రకం	ఎప్పుడు
B-Tree	Default; equality + range (`=` , `<` , `>` , BETWEEN, ORDER BY)
Hash	Equality మాత్రమే (`=`), range కాదు; చాలా fast point lookup
Bitmap	Low-cardinality columns (analytics/OLAP)
Full-text	Text search (`LIKE '%word%'` బదులు)
GiST/GIN (Postgres)	JSONB, arrays, geospatial, full-text

Key Points

- **Index = data కి sorted shortcut** → search O(n) నుండి O(log n). చాలా DBs **B+Tree** వాడతాయి (flat, wide → తక్కువ disk reads; leaves linked → range fast).
- **Clustered** (data physically ఆ order లో, ఒక్కటే, PK) vs **Non-clustered** (separate structure + pointers, చాలా).
- **Composite index column order matters** — leftmost prefix rule. Equality columns ముందు, range తర్వాత.
- **Covering index** = query columns అన్నీ index లో → index-only scan (table touch చేయదు).
- **Trade-off:** index reads fast కానీ writes slow + storage. High-selectivity, frequently-queried columns కి; low-selectivity / write-heavy కి జాగ్రత్త.
- **EXPLAIN** తో index వాడిందా (`Index Scan` good, `Seq Scan` = full scan bad) చూడు.

Interview దృష్టి

Q: Index ఎలా queries ని fast చేస్తుంది? దాని ధర ఏమిటి? A: Index ఒక sorted data structure (సాధారణంగా B+Tree) — data కి pointers ని sorted గా ఉంచుతుంది. దీంతో search full table scan O(n) బదులు O(log n) అవుతుంది (పుస్తకం index లాగా). **ధర:** ప్రతి INSERT/UPDATE/DELETE అన్ని affected indexes ని కూడా update చేయాలి → writes నెమ్మది; extra storage. అంటే index = read speed ని write speed + space తో కొనడం. Read-heavy కి worth, write-heavy లో select గా.

Q: Clustered vs Non-clustered index? A: **Clustered** — table rows ని physically index order లో store చేస్తుంది; అందుకే table కి **ఒక్కటే** (సాధారణంగా PK); data నేరుగా leaf లో. **Non-clustered** — separate structure, sorted keys + actual rows కి pointers; ఒక table కి **చాలా**; lookup కి extra hop (index → pointer → row). Analogy: clustered = పుస్తకం chapters order; non-clustered = చివర index.

Q: B-Tree ఎందుకు, binary tree కాదు? A: Databases disk మీద ఉంటాయి; disk read ఖరీదు. Binary tree ప్రతి node కి 2 children → tall tree → ఎక్కువ disk reads. **B+Tree** ప్రతి node కి 100s children → flat, wide tree → millions rows కి కేవలం 3-4 levels → చాలా తక్కువ disk reads. అదనంగా B+Tree లో అన్ని data leaves లో + leaves linked → range queries/sorting fast.

Q: Composite index `(a, b)` ఉంటే `WHERE b = 5` index వాడుతుందా? A: సాధారణంగా **లేదు** — leftmost prefix rule ప్రకారం composite index `a` తో మొదలయ్యే queries కి పనిచేస్తుంది. `WHERE a = ...` లేదా `WHERE a = ...`

AND b = ... వాడుతుంది; WHERE b = ... (a ని skip) సరిగ్గా వాడదు. అందుకే composite index లో column order design decision — equality/most-selective columns ముందు పెట్టాలి.

Q: ఒక query slow గా ఉంది, ఎలా debug చేస్తావు? A: `EXPLAIN ANALYZE` run చేసి execution plan చూస్తా. `Seq Scan` (full table scan) కనిపిస్తే → filter/join column మీద index లేదు అని అర్థం → index add చేస్తా. rows examined చాలా ఎక్కువ ఉంటే selectivity చూస్తా. Index ఉన్నా వాడకపోతే — column మీద function, leading wildcard, type mismatch, లేదా stale statistics కారణం అవ్వచ్చు. తర్వాత query rewrite / index tune చేస్తా.

13. Query Optimization & Storage

వివరణ

నువ్వు SQL query రాస్తావు — అది **ఏమి** కావాలో చెప్తుంది (declarative), **ఎలా** తేవాలో కాదు. Database లోని **query optimizer** ఆ "ఎలా" ని నిర్ణయిస్తుంది — ఏ index వాడాలి, ఏ join order, ఏ algorithm. అదే optimizer నీ query ని fast లేదా slow చేస్తుంది.

Query ఒక journey గా ఎలా process అవుతుంది:

```
SQL query text
      │
      ▼
1. Parser        → syntax check, parse tree తయారు
      │
      ▼
2. Optimizer     → చాలా possible execution plans generate చేసి, cost estimate చేసి, cheapest
ఎంచుకుంటుంది
      │            (ఏ index? ఏ join order? nested-loop/hash/merge join?)
      ▼
3. Executor      → ఎంచుకున్న plan ని run చేసి rows తెస్తుంది
      │
      ▼
   Results
```

Real-life Scenario

> **Query optimizer = Google Maps route planner.**
>
> నువ్వు "ఇంటి నుండి office కి వెళ్ళాలి" అని చెప్తావు (query — ఏమి కావాలో). Google Maps అనేక routes చూస్తుంది — highway, city roads, shortcut — ప్రతి దానికి **time/traffic estimate** చేసి, **cheapest (fastest)** route ఎంచుకుంటుంది. నువ్వు "ఏ route" అని చెప్పలేదు; అది నిర్ణయించింది.
>
> Query optimizer అదే — నువ్వు SQL లో "ఏమి కావాలో" చెప్తావు, optimizer అనేక execution plans (routes) ని cost తో compare చేసి fastest ఎంచుకుంటుంది. అందుకే అదే result కి, plan ని బట్టి 10ms లేదా 10s అవ్వచ్చు. **EXPLAIN = "ఏ route ఎంచుకున్నావు చూపించు."**

Query Execution Plan — cost-based optimization

Optimizer **cost-based** – ప్రతి plan కి estimated cost (disk reads + CPU) లెక్కించి, తక్కువ cost ఉన్నది ఎంచుకుంటుంది. ఈ estimate **statistics** మీద ఆధారపడుతుంది – table లో ఎన్ని rows, ఒక column లో ఎన్ని distinct values (cardinality), data distribution.

```
-- statistics update చేయడం (optimizer better decisions కి) — Postgres
ANALYZE users;
-- stale statistics → optimizer తప్పు plan (full scan when index better) → slow queries
```

Join algorithms — optimizer ఎంచుకునేవి:

Algorithm	ఎలా	ఎప్పుడు best
Nested Loop Join	ఒక table ప్రతి row కి, రెండో table search	ఒక table చిన్నది + join column మీద index
Hash Join	చిన్న table తో hash table build, పెద్ద table probe	పెద్ద tables, equality join, index లేదు
Merge Join	రెండు tables sort చేసి merge	రెండూ sorted (index) / sort చౌక

Optimizer data size + indexes బట్టి వీటిలో ఎంచుకుంటుంది. నువ్వు query ఎలా రాసినా, optimizer plan ని rewrite చేయవచ్చు.

Storage internals — Pages (blocks)

Database data disk మీద **pages (blocks)** గా store అవుతుంది – సాధారణంగా **8 KB** (Postgres) లేదా 16 KB (MySQL InnoDB). ఇది atomic unit of I/O – database ఒక row కావాలన్నా, ఆ row ఉన్న **మొత్తం page** ని disk నుండి memory లోకి తెస్తుంది.

ఎందుకు pages, individual rows కాదు? Disk I/O ఖరీదు – ఒక్క seek కి పెద్ద chunk చదవడం efficient (nearby rows కూడా వస్తాయి, తర్వాత అవి కావాలంటే already memory లో). ఒక page లో చాలా rows పడతాయి.

```
Disk లో table:
┌──────── Page 1 (8KB) ──────────┬──────── Page 2 (8KB) ──────────┬── ...
│ row1 | row2 | row3 | row4 ...│ row5 | row6 | row7 | ...        │
└──────────────────────────────┴────────────────────────────────┘
       ↑ database ఒక page ని unit గా read/write చేస్తుంది
```

Buffer Pool (Buffer Cache) — RAM లో pages caching

Buffer pool = database RAM లో ఉంచే memory area, ఇక్కడ recently-used pages cache అవుతాయి. ఎందుకు? **RAM disk కంటే ~100,000x fast.** Disk నుండి తెచ్చిన page ని buffer pool లో ఉంచితే, తర్వాత అదే page కావాలంటే disk కి వెళ్ళకుండా RAM నుండే → చాలా fast.

```
Query → page కావాలి?
         ├─ buffer pool లో ఉందా? ── YES ──> RAM నుండి (fast, "cache hit")
         └─ లేదు? ("cache miss") ──> disk నుండి read → buffer pool లో పెట్టు → return
```

- **Cache hit ratio** ఎక్కువ = better performance. Production DBs లో buffer pool size (Postgres: `shared_buffers`, MySQL: `innodb_buffer_pool_size`) tune చేయడం DBA పని.
- Buffer pool నిండితే, **eviction policy** (LRU — Least Recently Used variant) తో పాత pages తీసేస్తారు.
- **Dirty pages** (memory లో modified కానీ disk కి ఇంకా write కాని) ని background లో periodically disk కి flush చేస్తారు (checkpoint).

WAL (Write-Ahead Logging) — durability + performance రెండూ

WAL = ఏదైనా data change ని **actual data page కి రాయక ముందే**, ఒక **log file కి (sequential, append-only)** రాయడం. ఇది Topic 9 (Durability) లో చూశాం — ఇక్కడ deep.

ఎందుకు brilliant?

1. **Durability:** COMMIT అయ్యే ముందు change WAL కి (fast sequential write) flush → crash అయినా WAL నుండి replay చేసి recover. Data page ఇంకా disk కి వెళ్ళకపోయినా OK.
2. **Performance:** Data pages ని ప్రతి change కి disk కి రాయనవసరం లేదు (random writes, slow). Buffer pool లో collect చేసి, background లో batch గా flush. WAL మాత్రమే immediately రాయాలి — అది **sequential** (fast), random కాదు.

```
Change వచ్చింది:
1. WAL కి రాయి (sequential, fast) + flush        ← durability ఇక్కడే guaranteed
2. Buffer pool లో page ని modify (memory, fast)
3. తర్వాత ఎప్పుడో background లో data page ని disk కి flush (batched)

Crash అయితే → restart time లో WAL replay:
   - committed changes ని redo (data pages కి apply)
   - uncommitted ని undo
```

> **అంటే:** sequential log write (fast) + deferred random data write (batched) = durability + speed రెండూ. ఇది databases design లో అత్యంత elegant idea. (MySQL లో redo log, Oracle లో redo log — అదే concept.)

Query optimization — practical tips (SSE)

1. **SELECT * వాడొద్దు** — కావలసిన columns మాత్రమే (తక్కువ data, covering index possible).
2. **WHERE లో filter early** — తక్కువ rows process.
3. **Indexes** on WHERE/JOIN/ORDER BY columns (Topic 12).
4. **N+1 query problem తప్పించు** — loop లో query బదులు ఒక JOIN/ `IN`. (MERN/ORM లో common.)
5. **Column మీద function వద్దు** WHERE లో (`WHERE YEAR(date)=2024` → index బద్దలవుతుంది; `WHERE date >= '2024-01-01'` వాడు).
6. **LIMIT** పెద్ద results కి; keyset pagination.
7. **EXPLAIN ANALYZE** తో measure — guess చేయకు.
8. **Statistics fresh** గా ఉంచు (ANALYZE).

Key Points

- **Query journey:** Parser → **Optimizer** (cost-based, multiple plans compare) → Executor. నువ్వు "ఏమి" చెప్తావు, optimizer "ఎలా" నిర్ణయిస్తుంది.
- Optimizer **statistics** (row counts, cardinality) మీద ఆధారపడుతుంది; stale stats → bad plans. `ANALYZE` / `EXPLAIN` వాడు.
- **Join algorithms:** nested loop (చిన్న+index), hash join (పెద్ద, no index), merge join (sorted).
- **Storage:** data = **pages (8-16 KB)**, atomic I/O unit. **Buffer pool** = RAM లో pages cache (RAM ~100000x fast; cache hit ratio ముఖ్యం).
- **WAL** = change ని data page ముందు log కి (sequential) → durability (crash recovery) + performance (deferred batched data writes).

Interview దృష్టి

Q: SQL query database లో ఎలా process అవుతుంది? A: **Parser** syntax check చేసి parse tree తయారు చేస్తుంది. **Optimizer** అనేక possible execution plans (ఏ index, ఏ join order, ఏ join algorithm) generate చేసి, statistics ఆధారంగా ప్రతి plan cost estimate చేసి **cheapest ఎంచుకుంటుంది** (cost-based). **Executor** ఆ plan run చేసి rows తెస్తుంది. SQL declarative — నువ్వు "ఏమి కావాలో" చెప్తావు, optimizer "ఎలా తేవాలో" నిర్ణయిస్తుంది.

Q: Buffer pool అంటే ఏమిటి, ఎందుకు? A: Database RAM లో ఉంచే cache area — recently-accessed disk pages ని store చేస్తుంది. RAM disk కంటే ~100,000x fast కాబట్టి, ఒకసారి తెచ్చిన page ని memory లో ఉంచితే తర్వాత అదే page కి disk I/O తప్పుతుంది (cache hit). Cache hit ratio ఎక్కువ = better performance. నిండితే LRU తో పాత pages evict చేస్తారు. Buffer pool size tuning production DB performance కి కీలకం.

Q: WAL ఎలా durability + performance రెండూ ఇస్తుంది? A: Change ని ముందు **WAL (sequential, append-only log)** కి రాసి flush చేస్తారు — ఇది fast (sequential write) మరియు durability guarantee (crash అయినా log replay చేసి recover). Actual data pages ని వెంటనే disk కి రాయనవసరం లేదు — buffer pool లో collect చేసి background లో batch గా flush (random writes deferred). అంటే durability కోసం sequential log write మాత్రమే critical path లో; slow random data writes deferred → speed. Crash అయితే WAL replay: committed redo, uncommitted undo.

Q: N+1 query problem అంటే ఏమిటి? A: Parent records 1 query లో తెచ్చి, తర్వాత ప్రతి parent కి children ని separate query లో తేవడం — 1 (parents) + N (ప్రతి parent కి) = N+1 queries. ఉదా: 100 orders తెచ్చి, ప్రతి order కి customer ని విడిగా query చేస్తే 101 queries. Fix: JOIN లేదా `WHERE id IN (...)` తో ఒకే query. ORMs (Mongoose populate, Sequelize) లో common trap — eager loading / batching తో solve.

Part 6 — Modern (ఆధునికం)

> నీకు ఇప్పటికే MongoDB (NoSQL) తెలుసు. ఇప్పుడు relational foundations కూడా తెలుసు. ఈ Part లో ఆ రెండింటినీ కలిపి — **SQL vs NoSQL ఎప్పుడు ఏది**, databases ని ఎలా **scale** చేస్తారు (replication, sharding, CAP theorem), చివరగా అన్నీ కలిపిన **interview Q&A + memory tips + common mistakes** — చూద్దాం. ఇది నీ MERN అనుభవాన్ని theory తో కలిపి, interview లో confident గా మాట్లాడేలా చేస్తుంది.

---

14. SQL vs NoSQL

వివరణ

SQL (Relational) databases — data ని tables (rows/columns), fixed schema, relationships (FK), joins. ACID strong. ఉదా: PostgreSQL, MySQL, Oracle.

NoSQL ("Not Only SQL") databases — non-relational, flexible schema, horizontal scaling కి designed. Data ని documents/key-value/columns/graphs గా store చేస్తాయి. ఉదా: **MongoDB (document)**, Redis (key-value), Cassandra (wide-column), Neo4j (graph).

నీ MERN world లో: M = **MongoDB** = NoSQL document DB. నువ్వు ఇప్పటివరకు NoSQL వాడావు. ఈ topic నీకు "relational world ఎందుకు, ఎప్పుడు better" అని చూపిస్తుంది — interview లో "SQL vs NoSQL, ఎప్పుడు ఏది" అనేది దాదాపు guaranteed question.

Real-life Scenario

> **SQL vs NoSQL = form-filling office vs whiteboard startup.**
>
> - **SQL = government office.** ప్రతి form fixed format — అన్ని columns నింపాలి, rules కఠినం (schema, constraints). కొంచెం rigid కానీ **అంతా consistent, verified, connected** (ఒక file మరో file ని reference చేస్తుంది — joins). Data integrity అత్యంత ముఖ్యమైన చోట (bank, tax) ఇదే కావాలి.
> - **NoSQL = startup whiteboard.** ఎవరు ఏమైనా రాయొచ్చు, format flexible (schema-less). వేగంగా మార్చొచ్చు, పెద్దగా scale అవుతుంది. కానీ "ఈ data అన్నిచోట్లా consistent గా ఉందా?" అనేది నీ బాధ్యత.
>
> రెండూ మంచివే — **సందర్భాన్ని బట్టి.** Bank transactions కి government office (SQL); social media feed, product catalog కి flexible whiteboard (NoSQL).

SQL vs NoSQL — పూర్తి comparison (interview core)

అంశం	SQL (RDBMS)	NoSQL (ఉదా: MongoDB)
Data model	Tables (rows/columns)	Documents (JSON/BSON), key-value, column, graph
Schema	Fixed, predefined (ALTER అవసరం)	Flexible/dynamic (document కి document వేరు)
Relationships	Foreign keys + **joins**	Embedding లేదా referencing (weak joins)
Query language	SQL (standard, powerful)	DB-specific (Mongo query API, `$lookup`)
Transactions	Strong ACID (multi-row)	Eventually consistent; Mongo 4.0+ multi-doc ACID (limited)
Scaling	Vertical (బలమైన server) ప్రధానం; sharding కష్టం	**Horizontal** (add servers) native — sharding easy
Consistency	Strong (ACID)	Tunable (often eventual — BASE)
Best for	Structured data, complex queries, integrity	Large scale, rapid change, semi-structured
ఉదాహరణలు	Banking, ERP, inventory, orders	Catalog, IoT, real-time feeds, logs, caching

ACID vs BASE

SQL = **ACID** (Atomicity, Consistency, Isolation, Durability — strict correctness). NoSQL తరచుగా = **BASE:**

- **B**asically **A**vailable — ఎప్పుడూ respond అవుతుంది (partial data అయినా).
- **S**oft state — state కాలంతో మారొచ్చు (background sync).
- **E**ventual consistency — వెంటనే కాకపోయినా, చివరికి అన్ని nodes consistent అవుతాయి.

Trade-off: ACID = correctness ప్రధానం (కొంచెం slow/less available); BASE = availability + scale ప్రధానం (కొంత consistency lag).

NoSQL రకాలు — 4 categories

రకం	ఏమిటి	ఉదాహరణ	ఎప్పుడు
Document	JSON-like documents, nested	**MongoDB**, CouchDB	Catalog, user profiles, CMS, MERN apps
Key-Value	Simple key → value map	**Redis**, DynamoDB	Caching, sessions, rate limiting, leaderboards
Wide-Column	Rows with dynamic columns, huge scale	**Cassandra**, HBase	Time-series, IoT, write-heavy at massive scale
Graph	Nodes + edges (relationships)	**Neo4j**	Social networks, recommendations, fraud detection

MongoDB deep — నీకు తెలిసిన దానికి theory connect

నీకు తెలిసిన Mongo concepts ని relational పదాలతో map చేద్దాం:

MongoDB	SQL సమానం	గమనిక
Database	Database	same
Collection	Table	schema-less
Document	Row	BSON (binary JSON)
Field	Column	document కి document వేరు అవ్వొచ్చు
Embedded document	(denormalized join)	nested data ఒకే document లో
`$lookup`	JOIN	slower, less powerful
`_id` (ObjectId)	Primary key	auto-generated
Index	Index	Mongo కూడా B-tree indexes
Sharding (built-in)	Manual sharding	Mongo native horizontal scale

Embedding vs Referencing (Mongo design — నువు రోజూ చేసేది):

- **Embedding:** related data ని ఒకే document లో (order లోపల items array). → **read fast** (ఒక query, no join), కానీ duplication + document size limit (16MB). "Read together, store together" కి.
- **Referencing:** ObjectId తో link (SQL FK లాగా). → normalized, no duplication, కానీ multiple queries / `$lookup` . Many-to-many, large/shared data కి.

ఇదే relational normalization vs denormalization decision — Mongo లో మీరు దీన్ని manually చేస్తారు; SQL default normalized + joins.

ఎప్పుడు SQL, ఎప్పుడు NoSQL? (decision guide — SSE)

SQL ఎంచుకో:

- Data structured, relationships ముఖ్యం (orders, users, payments).
- **Strong consistency / ACID transactions** అవసరం (banking, inventory, bookings).
- Complex queries, joins, reporting, analytics.
- Schema stable.

NoSQL ఎంచుకో:

- Schema flexible / తరచుగా మారుతుంది (rapid product iteration).
- **Massive horizontal scale** (millions writes/sec, huge data).
- Semi-structured / hierarchical data (JSON — catalogs, profiles, IoT, logs).
- Read-heavy, denormalized access patterns; joins అవసరం లేదు.
- Specific: caching (Redis), real-time (feeds), time-series.

> **Polyglot persistence (SSE-level insight):** Real systems తరచుగా **రెండూ** వాడతాయి. ఉదా: ఒక e-commerce app — orders/payments **PostgreSQL** (ACID), product catalog **MongoDB** (flexible), sessions/cart **Redis** (fast), search **Elasticsearch**. "ఒకటే database అన్నిటికీ" అనేది naive; **right tool for each job.** Interview లో ఇది చెప్తే maturity కనిపిస్తుంది.

సాధారణ అపోహలు (myths)

- ❌ "NoSQL ఎప్పుడూ faster" — కాదు; access pattern బట్టి. Joins అవసరమైతే SQL faster.
- ❌ "SQL scale అవ్వదు" — కాదు; sharding/read replicas తో పెద్ద SQL systems ఉన్నాయి (కొంచెం కష్టం మాత్రమే).
- ❌ "NoSQL కి schema లేదు" — schema-flexible, కానీ implicit schema application లో ఉంటుంది (Mongoose schemas!).
- ❌ "NoSQL కి ACID లేదు" — పాత మాట; MongoDB 4.0+ multi-document ACID transactions ఇస్తుంది.

Key Points

- **SQL** = tables, fixed schema, joins, strong **ACID**, vertical scaling ప్రధానం. **NoSQL** = flexible schema, horizontal scaling, tunable consistency (**BASE**).
- **NoSQL 4 రకాలు:** document (MongoDB), key-value (Redis), wide-column (Cassandra), graph (Neo4j).
- **Mongo mapping:** collection=table, document=row, field=column, `$lookup` =join (weaker), sharding built-in.
- **Embedding (read fast, denormalized) vs referencing (normalized, joins)** = Mongo లో normalization decision.
- **SQL:** structured + integrity + transactions + complex queries. **NoSQL:** flexible + massive scale + semi-structured.
- **Polyglot persistence** — real systems రెండూ వాడతాయి; right tool per job.

Interview దృష్టి

Q: SQL vs NoSQL, ఎప్పుడు ఏది ఎంచుకుంటావు? A: **SQL** — data structured, relationships + joins ముఖ్యం, **strong ACID consistency** అవసరం (banking, orders, inventory, bookings), complex queries/reporting. **NoSQL** — schema flexible/మారుతుంది, **massive horizontal scale** అవసరం, semi-structured data (JSON — catalogs, profiles, logs, IoT), denormalized read patterns. చాలా real systems **రెండూ వాడతాయి (polyglot persistence)** — ఉదా: transactions కి Postgres, catalog కి Mongo, cache కి Redis. Requirement (consistency vs scale vs flexibility) బట్టి decide చేస్తా.

Q: ACID vs BASE? A: **ACID** (SQL) — Atomicity, Consistency, Isolation, Durability; strict correctness, strong consistency, transactions. **BASE** (చాలా NoSQL) — Basically Available, Soft state, Eventual consistency; availability + scale కి consistency ని కొంచెం loosen చేస్తుంది (వెంటనే కాక చివరికి consistent). Bank transfer కి ACID; social media likes count కి eventual consistency (BASE) చాలు.

Q: MongoDB లో embedding vs referencing ఎప్పుడు? A: **Embedding** (nested document) — related data ఎప్పుడూ కలిసి read అయ్యేది, one-to-few, ownership ఉన్నది (order + its items). Read fast (ఒక query, no join),

కానీ duplication + 16MB doc limit. **Referencing** (ObjectId link, SQL FK లా) — many-to-many, large/shared/independently-queried data, unbounded growth. Normalized, కానీ `$lookup` /multiple queries. ఇది SQL normalization vs denormalization decision యొక్క Mongo రూపం.

Q: "NoSQL scale అవుతుంది, SQL అవ్వదు" నిజమా? A: పూర్తిగా నిజం కాదు. NoSQL **horizontal scaling (sharding) ని native గా, సులభంగా** support చేస్తుంది — అదే దాని బలం. SQL కూడా scale అవుతుంది (read replicas, sharding, partitioning) కానీ joins/transactions వల్ల sharding కష్టం. అంటే NoSQL scale-out easier, SQL scale-out possible but harder. మరోవైపు SQL strong consistency + complex queries లో superior. Trade-off — scale vs consistency/query power.

15. Scaling: Replication, Sharding, CAP Theorem

వివరణ

ఒక database server కి limit ఉంది — ఒక machine ఎంత RAM, CPU, disk handle చేయగలదో అంతే. Users/data పెరిగితే ఒకే server సరిపోదు. **Scaling** = ఎక్కువ load ని handle చేయడానికి database capacity పెంచడం. రెండు మార్గాలు:

రకం	అర్థం	Analogy
Vertical Scaling (scale up)	ఉన్న server ని పెద్దది చేయడం (ఎక్కువ RAM/CPU)	ఒక ట్రక్కుని పెద్దది కొనడం
Horizontal Scaling (scale out)	ఎక్కువ servers add చేయడం, load పంచడం	ఎక్కువ ట్రక్కులు కొనడం

Vertical simple కానీ hardware limit + single point of failure. Horizontal complex కానీ unlimited scale + fault tolerance — పెద్ద systems దీన్నే వాడతాయి.

> **గమనిక:** ఇది database దృష్టికోణం నుండి **brief overview.** Load balancing, caching layers, CDN, microservices scaling లాంటి full system design depth కోసం `HLD_Go_Telugu.md` మరియు `SystemDesign_Go_Telugu.md` చూడు — అక్కడ ఇవి చాలా లోతుగా ఉన్నాయి.

Real-life Scenario

> **Replication vs Sharding = restaurant scaling.**
>
> నీ restaurant కి customers పెరిగారు:
>
> - **Replication (copies):** అదే menu ఉన్న **అనేక identical branches** తెరవడం. ఏ branch కి వెళ్ళినా అదే food. ఒక branch మూసినా మిగతావి పనిచేస్తాయి. **అదే data అనేక copies** — read load పంచుతుంది, fault-tolerant.
> - **Sharding (split):** ఒకే పెద్ద kitchen ని **విభాగాలుగా** విడగొట్టడం — ఒక counter South Indian, ఒకటి North Indian, ఒకటి Chinese. ప్రతి counter **వేరే data** handle చేస్తుంది. కలిసి full menu; విడివిడిగా load తక్కువ.
>
> Replication = అదే data ని copy చేయడం (read scale + reliability). Sharding = వేరే data ని వేరే servers కి పంచడం (write scale + huge data). పెద్ద systems **రెండూ కలిపి** వాడతాయి.

Replication — data ని multiple copies గా ఉంచడం

Replication = అదే data ని అనేక servers (replicas) లో copy చేయడం. Primary (leader) కి writes, replicas (followers) కి copy.

```
          writes             replication (copy)
Client ──────────▶  ┌────────┐  ──────────▶  ┌──────────┐   ◀── reads
                    │PRIMARY │  ──────────▶  │REPLICA 1│
                    │(leader)│               └──────────┘
                    └────────┘  ──────────▶  ┌──────────┐   ◀── reads
                                             │REPLICA 2│
                                             └──────────┘
```

Benefits:

- **Read scaling:** reads ని replicas కి పంచడం (read-heavy apps కి huge win).
- **High availability / fault tolerance:** primary fail అయితే, ఒక replica ని promote చేసి కొనసాగించడం (failover).
- **Geo-distribution:** users దగ్గర replica → తక్కువ latency.

రకాలు:

- **Synchronous:** primary, replica confirm అయ్యేవరకు wait → strong consistency, కానీ slow.
- **Asynchronous:** primary వెంటనే respond, replica తర్వాత update → fast, కానీ **replication lag** (replica కొద్దిసేపు stale). చాలా systems async వాడతాయి.

> **Replication lag gotcha:** async replication లో, ఒక user data write చేసి వెంటనే read చేస్తే (వేరే replica నుండి), పాత data రావచ్చు ("నేను post చేశాను కానీ కనిపించట్లేదు"). Solution: "read-your-own-writes" — critical reads ని primary నుండి.

Sharding (Partitioning) — data ని విభజించడం

Sharding = data ని అనేక servers (shards) లో **విభజించడం** — ప్రతి shard **వేరే subset** of data. ఒక **shard key** (ఉదా: user_id, region) ఆధారంగా ఏ row ఏ shard కి వెళ్తుందో నిర్ణయిస్తారు.

```
                 shard key (ఉదా: user_id % 3)
Client ──▶ Router ──┬──▶ Shard 0 (users 0,3,6...)
                    ├──▶ Shard 1 (users 1,4,7...)
                    └──▶ Shard 2 (users 2,5,8...)
```

Benefits: write scaling (writes పంచబడతాయి), huge datasets (ఒక server లో పట్టనంత data), తక్కువ per-shard load.

Challenges (interview లో అడుగుతారు):

- **Cross-shard queries/joins కష్టం** (data వేర్వేరు servers లో) → slow, complex.
- **Shard key choice కీలకం** — bad key → **hotspots** (ఒక shard overload). ఉదా: timestamp key → కొత్త data అంతా ఒకే shard కి.

- **Rebalancing కష్టం** — shard add చేస్తే data move చేయాలి.
- **Transactions across shards** చాలా కష్టం.

Partitioning రకాలు: range-based (A-M shard 1, N-Z shard 2), hash-based (hash(key) % N — even distribution), directory-based (lookup table).

Vertical vs Horizontal partitioning:

- **Horizontal (sharding):** rows ని పంచడం (users 1-1000 shard A, 1001-2000 shard B).
- **Vertical:** columns ని పంచడం (frequently-used columns ఒక table, rarely-used మరో table).

CAP Theorem — distributed DB యొక్క ప్రాథమిక పరిమితి

CAP Theorem (Eric Brewer): ఒక distributed system లో ఈ **3 లో గరిష్ఠంగా 2 మాత్రమే** ఏకకాలంలో guarantee చేయగలవు:

అక్షరం	Property	అర్థం
C	**Consistency**	ప్రతి read అత్యంత recent write ని చూస్తుంది (అన్ని nodes same data)
A	**Availability**	ప్రతి request కి response వస్తుంది (node down అయినా)
P	**Partition Tolerance**	Network split (nodes మధ్య communication fail) అయినా system పనిచేస్తుంది

కీలక insight: Distributed system లో **network partitions అనివార్యం** (P ని వదులుకోలేం) → అసలు choice **C vs A** (partition సమయంలో):

```
         Consistency (C)
             /\
            /  \
           / CP \  ← partition లో consistency కోసం availability వదులు
          /______\    (కొన్ని requests fail/wait) — ఉదా: RDBMS, MongoDB, HBase
         /\      /\
        /  \    /  \
       / CA \  / AP \ ← partition లో availability కోసం consistency వదులు
      /______\/______\   (stale data ok) — ఉదా: Cassandra, DynamoDB, CouchDB
  Availability(A)  Partition Tolerance(P)
  (CA = single node only, real distributed లో అసాధ్యం)
```

- **CP systems** (Consistency + Partition tolerance): partition అయితే, consistency కోసం కొన్ని requests reject/wait. ఉదా: **traditional RDBMS (distributed), MongoDB, HBase, Redis.** Banking లాంటి correctness ముఖ్యమైన చోట.
- **AP systems** (Availability + Partition tolerance): partition అయినా respond అవుతుంది, కానీ stale data రావొచ్చు (eventual consistency). ఉదా: **Cassandra, DynamoDB, CouchDB.** Social feeds, shopping cart లాంటి availability ముఖ్యమైన చోట.
- **CA** — network partition లేని single-node systems మాత్రమే; నిజమైన distributed system లో impossible (P తప్పనిసరి).

> **PACELC (advanced):** CAP ని విస్తరించి — partition ఉన్నప్పుడు C vs A; **partition లేనప్పుడు (Else) Latency vs Consistency.** అంటే normal operation లో కూడా consistency కి latency price ఉంది. (`SystemDesign_Go_Telugu.md` లో deep.)

Key Points

- **Vertical scaling** (server పెద్దది, simple, limit) vs **Horizontal scaling** (ఎక్కువ servers, complex, unlimited).
- **Replication** = అదే data multiple copies → **read scaling + high availability + failover.** Sync (consistent, slow) vs async (fast, replication lag).
- **Sharding** = data ని shards గా విభజన (shard key) → **write scaling + huge data.** Challenges: cross-shard joins, hotspots, rebalancing.
- **CAP theorem:** distributed system లో C, A, P లో 2 మాత్రమే. P అనివార్యం → **CP** (consistency, ఉదా RDBMS/Mongo) vs **AP** (availability, ఉదా Cassandra/DynamoDB).
- Full system-design depth: `HLD_Go_Telugu.md` , `SystemDesign_Go_Telugu.md` .

Interview దృష్టి

Q: Replication vs Sharding తేడా? A: **Replication** = అదే data ని multiple servers లో copy చేయడం — read scaling, high availability, failover ఇస్తుంది (primary fail → replica promote). **Sharding** = data ని shard key ఆధారంగా వేర్వేరు servers లో విభజించడం (ప్రతి shard వేరే subset) — write scaling, huge datasets handle చేస్తుంది. Replication = redundancy (same data copies); sharding = distribution (different data pieces). పెద్ద systems రెండూ కలిపి వాడతాయి (sharded + each shard replicated).

Q: CAP theorem వివరించు. A: Distributed system లో Consistency (అన్ని nodes same recent data), Availability (ప్రతి request కి response), Partition tolerance (network split అయినా పనిచేయడం) — ఈ 3 లో గరిష్ఠంగా 2 మాత్రమే guarantee చేయగలం. Network partitions అనివార్యం కాబట్టి P తప్పనిసరి → అసలు choice **C vs A**. **CP** systems (RDBMS, MongoDB) partition లో consistency కోసం availability త్యాగం చేస్తాయి; **AP** systems (Cassandra, DynamoDB) availability కోసం stale data (eventual consistency) accept చేస్తాయి. Use-case బట్టి choose (banking → CP, feed → AP).

Q: Sharding లో shard key ఎలా ఎంచుకుంటావు? A: Shard key data ని **సమానంగా distribute** చేయాలి (hotspots రాకుండా) మరియు common queries ని ఒకే shard లో ఉంచాలి (cross-shard queries తగ్గించడానికి). Bad key ఉదా: timestamp → కొత్త data అంతా ఒకే shard కి (hotspot). Good: high-cardinality, even distribution ఉన్న key (hash(user_id)). Trade-off: query locality vs even distribution. Choice మార్చడం చాలా కష్టం (rebalancing) కాబట్టి ముందే జాగ్రత్త.

Q: Replication lag అంటే ఏమిటి, ఎలా handle చేస్తావు? A: Async replication లో primary write అయిన తర్వాత replicas update అవ్వడానికి కొంత delay — ఈ మధ్య replica నుండి read చేస్తే **stale (పాత) data** వస్తుంది. Handle: (1) critical/just-written reads ని **primary నుండి** (read-your-own-writes), (2) sync replication (కానీ slow), (3) application లో eventual consistency ని accept చేయగల చోట్ల మాత్రమే replicas నుండి read. Trade-off: consistency vs read-scaling.

16. Interview Q&A + Memory Tips + Common Mistakes

వివరణ

ఈ చివరి topic = మొత్తం guide యొక్క **revision + interview weapon.** ఇక్కడ rapid-fire Q&A, ప్రతి concept ని జీవితంలో మర్చిపోని memory hooks, మరియు అందరూ చేసే mistakes ఉన్నాయి. Interview ముందు రోజు ఇది చదివితే చాలు.

Real-life Scenario

> **ఈ section = exam ముందు రోజు "important questions" sheet.** పుస్తకం మొత్తం చదివాక, చివర్లో ఒక్క page లో అన్ని key points ఉంటే – గుర్తుకు తెచ్చుకోవడం సులభం. Interview hall లోకి వెళ్ళే ముందు ఇది ఒక్కసారి కళ్ళతో చదువు.

Rapid-Fire Interview Q&A (అత్యధికంగా అడిగేవి)

Q1: DELETE vs TRUNCATE vs DROP? DELETE = DML, WHERE తో selective rows, rollback ok, slow. TRUNCATE = DDL, అన్ని rows fast, WHERE లేదు, identity reset. DROP = table structure+data మొత్తం పోతుంది.

Q2: Primary key vs Unique key? PK: NOT NULL + UNIQUE, ఒక్కటే per table. Unique: ఒక NULL allow, చాలా ఉండొచ్చు.

Q3: WHERE vs HAVING? WHERE rows ని grouping ముందు filter (aggregate వాడలేం). HAVING groups ని grouping తర్వాత (aggregate వాడొచ్చు).

Q4: INNER vs LEFT JOIN? INNER = match ఉన్నవి మాత్రమే. LEFT = ఎడమ table అంతా + కుడి match (లేకపోతే NULL).

Q5: ACID? Atomicity (all-or-nothing), Consistency (valid→valid), Isolation (concurrent = serial-లా), Durability (COMMIT = శాశ్వతం).

Q6: Normalization ఎందుకు? Redundancy + 3 anomalies (insert/update/delete) తగ్గించడానికి. 1NF (atomic) → 2NF (no partial dep) → 3NF (no transitive dep) → BCNF.

Q7: Index trade-off? Reads fast (O(log n), B+Tree), కానీ writes slow + storage. High-selectivity, frequently-queried columns కి.

Q8: Clustered vs Non-clustered index? Clustered = data physically ఆ order (ఒక్కటే, PK). Non-clustered = separate + pointers (చాలా).

Q9: SQL vs NoSQL ఎప్పుడు? SQL = structured + ACID + joins + integrity. NoSQL = flexible schema + horizontal scale + semi-structured. Real systems రెండూ (polyglot).

Q10: Dirty read, non-repeatable read, phantom read? Dirty = uncommitted చదవడం. Non-repeatable = అదే row వేరే value. Phantom = కొత్త rows కనిపించడం. Isolation levels: READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE.

Q11: Deadlock ఎలా handle? Detection (wait-for graph cycle → victim rollback → retry) లేదా prevention (consistent lock ordering).

Q12: CAP theorem? Distributed లో C, A, P లో 2 మాత్రమే. P అనివార్యం → CP (RDBMS/Mongo) vs AP (Cassandra/DynamoDB).

Q13: Replication vs Sharding? Replication = same data copies (read scale + HA). Sharding = data split by shard key (write scale + huge data).

Q14: Foreign key ఏం చేస్తుంది? Referential integrity — FK ఎప్పుడూ valid PK ని point చేయాలి. ON DELETE CASCADE/SET NULL/RESTRICT.

Q15: Query slow — ఎలా debug? EXPLAIN ANALYZE → Seq Scan (index లేదు) కనిపిస్తే index add. Statistics fresh, SELECT * తప్పించు, N+1 తప్పించు.

Q16: WAL ఎందుకు? Durability (crash recovery) + performance (sequential log write + deferred batched data writes).

Q17: Subquery vs JOIN? తరచుగా same result; JOIN సాధారణంగా faster (optimizer better handle). Correlated subquery slow (per-row).

Q18: Window function vs GROUP BY? GROUP BY rows collapse. Window aggregate + rows ఉంచుతుంది (ranks, running totals, top-N per group).

Memory Tips Table — జీవితంలో మర్చిపోని hooks

Concept	గుర్తుంచుకునే hook
ACID	**A**ll-or-nothing, **C**orrect, **I**solated, **D**urable — bank transfer story
3NF	"The **key**, the **whole key**, and **nothing but** the key"
1NF/2NF/3NF	1=atomic, 2=**whole** key (partial వద్దు), 3=**only** key (transitive వద్దు)
WHERE vs HAVING	WHERE = **rows ముందు**, HAVING = **groups తర్వాత**
INNER/LEFT	INNER = **ఉమ్మడి** (∩), LEFT = **ఎడమ అంతా**
Index	పుస్తకం చివర **index** — data కి shortcut, కానీ write cost
B+Tree	**flat & wide** → తక్కువ disk reads; leaves **linked** → range fast
Clustered	table యొక్క **actual order** (ఒక్కటే); non-clustered = **pointers** (చాలా)
Isolation levels	UNCOMMITTED→COMMITTED→REPEATABLE→SERIALIZABLE (safe↑ speed↓)
Dirty/Non-rep/Phantom	dirty=**uncommitted**, non-rep=**value మారింది**, phantom=**rows మారాయి**
Deadlock	ఇరుకు వంతెన ఎదురెదురు కార్లు → ఒకరు వెనక్కి (victim)
CAP	partition లో **C లేదా A** — banking=C, feed=A
Replication/Sharding	replicate=**copy** (read), shard=**split** (write)
WAL	log **ముందు**, data page **తర్వాత** (write-**ahead**)
DECIMAL for money	FLOAT = **approximate** → 0.1+0.2≠0.3; DECIMAL = exact
NULL	"**unknown**" — `IS NULL` వాడు, `= NULL` కాదు
SQL vs NoSQL	SQL=**govt office** (rules), NoSQL=**startup whiteboard** (flexible)

Common Mistakes — అందరూ చేసేవి (interview + production)

తప్పు	ఎందుకు ప్రమాదం	సరైనది
`UPDATE`/`DELETE` లో **WHERE మర్చిపోవడం**	మొత్తం table మారుతుంది/పోతుంది	ముందు `SELECT` తో verify, transaction లో run
Money కి FLOAT	rounding errors (0.1+0.2≠0.3)	`DECIMAL(p,s)`
`WHERE col = NULL`	ఏ row రాదు (NULL=NULL → unknown)	`IS NULL` / `IS NOT NULL`
`col <> 'x'` NULL rows ని వదిలేయడం	silently missing data	`col <> 'x' OR col IS NULL`
SELECT * everywhere	అనవసర data, covering index పోతుంది	కావలసిన columns మాత్రమే
N+1 queries (loop లో query)	100 orders → 101 queries, slow	JOIN / `IN (...)` / eager load
Over-indexing	ప్రతి write అన్ని indexes update → slow writes	అవసరమైన indexes మాత్రమే
Under-indexing (WHERE/JOIN columns)	full table scans, slow reads	frequently-queried columns కి index
GROUP BY లో **non-aggregate SELECT column**	error / wrong result	GROUP BY లో పెట్టు లేదా aggregate
`LIKE '%text'` (leading wildcard)	index వాడదు, full scan	trailing `'text%'` / full-text search
No transaction for multi-step writes	partial updates, inconsistency	`BEGIN...COMMIT` , atomicity
Auto-commit తెలియకపోవడం	ROLLBACK పనిచేయదు	explicit `BEGIN`
Large OFFSET pagination	OFFSET 1000000 → slow	keyset pagination (`WHERE id > last`)
ON DELETE CASCADE blindly	పొరపాటున parent delete → children silent loss	RESTRICT / soft-delete for critical data
Function on indexed column in WHERE	index బద్ధలవుతుంది (`YEAR(date)=2024`)	range (`date >= '2024-01-01'`) / functional index
Ignoring EXPLAIN	guess-based optimization	EXPLAIN ANALYZE తో measure

Interview లో ఎలా present చేయాలి (SSE communication)

1. **Requirement clarify చేయి** — "consistency ఎంత ముఖ్యం? scale ఎంత? read-heavy/write-heavy?" ముందు అడుగు. Blindly answer చేయకు.
2. **Trade-offs మాట్లాడు** — ప్రతి decision కి "ఇది fast కానీ ఇది cost" అని రెండువైపులా చెప్పు. Senior signal ఇదే.

3. **Real-world connect చెయ్** — "Mongo లో నేను ఇలా చేశాను, SQL లో ఇలా" — నీ MERN అనుభవాన్ని leverage చెయ్.
4. **Numbers/scale ప్రస్తావించు** — "10 lakh rows అయితే index లేకుండా O(n) scan..."
5. **Whiteboard schema/ER** — design questions లో tables, keys, relationships గీయి.

చివరి మాట (final revision path)

```
నేర్చుకున్న ప్రయాణం:
Part 1 (Basics)   → DBMS ఎందుకు, ER model, keys — పునాది
Part 2 (SQL)      → CRUD, SELECT, joins, aggregation — రోజువారీ ఆయుధం
Part 3 (Design)   → normalization — clean schema
Part 4 (Txns)     → ACID, isolation, deadlocks — correctness under concurrency
Part 5 (Perf)     → indexing, optimizer, WAL — speed at scale
Part 6 (Modern)   → SQL vs NoSQL, scaling, CAP — big-picture judgment
```

SSE interview లో నిన్ను వేరుగా నిలబెట్టేది: syntax కాదు — **"ఎప్పుడు ఏది, ఎందుకు, ఏ trade-off"** అనే judgment. ఈ guide లో ప్రతి topic లో అదే నొక్కి చెప్పాను. ఒకసారి కాదు, రెండు-మూడు సార్లు చదువు; SQL queries నిజంగా run చేయి; ప్రతి analogy ని నీ మాటల్లో ఒకరికి చెప్పు. అప్పుడు **"ఒకసారి చదివితే జీవితంలో మర్చిపోవు."**

Key Points

- **DELETE/TRUNCATE/DROP, PK/Unique, WHERE/HAVING, INNER/LEFT, ACID, normalization, index trade-off, isolation levels, CAP, replication/sharding** — ఇవి guaranteed interview questions. ఒక్క వాక్యంలో చెప్పగలగాలి.
- **Common mistakes:** WHERE మర్చిపోవడం, money కి FLOAT, `= NULL`, SELECT *, N+1, over/under-indexing, no transaction, large OFFSET.
- **Communication:** clarify → trade-offs → real-world connect → numbers → whiteboard. Senior signal = judgment, syntax కాదు.

Interview దృష్టి

Q: ఒక new feature కి database ఎలా design చేస్తావు (end-to-end)? A: (1) **Requirements clarify** — data ఏమిటి, access patterns (read/write ratio), scale, consistency needs. (2) **ER model** — entities, relationships, cardinality గీయడం. (3) **Tables + keys** — PK/FK, constraints, normalize to 3NF. (4) **Indexes** — WHERE/JOIN/ORDER BY columns మీద. (5) **Transactions** — multi-step writes కి ACID boundaries. (6) **SQL vs NoSQL** decide — structured+consistency=SQL, flexible+scale=NoSQL, అవసరమైతే both. (7) **Scale plan** — read replicas, sharding అవసరమైతే. ప్రతి step లో trade-offs చెప్పడం.

Q: నీ MERN అనుభవం DBMS interview కి ఎలా help చేస్తుంది? A: MongoDB వాడటం వల్ల documents, indexing, embedding vs referencing, schema design (Mongoose) practical గా తెలుసు. ఈ guide ఆ practical knowledge కి relational theory (normalization, ACID, joins, isolation) జోడించింది. Interview లో "Mongo లో ఇలా, SQL లో ఎందుకు వేరుగా, ఎప్పుడు ఏది" అని compare చేసి మాట్లాడగలను — ఇది ఒక్క database మాత్రమే తెలిసిన వాళ్ళ కంటే broader perspective, SSE level కి కావలసిన judgment చూపిస్తుంది.

Q: Production లో slow queries ఎలా systematically fix చేస్తావు? A: (1) **Identify** — slow query log / APM తో ఏ queries slow గుర్తించడం. (2) **EXPLAIN ANALYZE** — execution plan చూడడం (Seq Scan? high rows? bad join?). (3) **Index** — missing indexes add (WHERE/JOIN/ORDER BY). (4) **Query rewrite** — SELECT * తీయడం, N+1 → JOIN, correlated subquery → JOIN/window, function-on-column తీయడం. (5) **Schema** — అవసరమైతే denormalize (read-heavy), partitioning. (6) **Caching** — repeated reads కి Redis. (7) **Scale** — read replicas. Measure before/after, guess చేయకుండా data-driven.